

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ**  
**«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ**  
**імені Ігоря СІКОРСЬКОГО»**  
**Навчально-науковий фізико-технічний інститут**  
**Кафедра математичного моделювання та аналізу даних**

«На правах рукопису»

УДК 004.93:004.8

«До захисту допущено»

В.о. зав. кафедри

\_\_\_\_\_ Іван ТЕРЕЩЕНКО

«\_\_\_» \_\_\_\_\_ 2026 р.

**Дипломна робота**  
**на здобуття ступеня бакалавра**  
**за освітньо-професійною програмою**  
**«Математичні методи моделювання, розпізнавання образів та комп'ютерного**  
**зору»**

зі спеціальності: 113 Прикладна математика  
на тему: **«Інтерактивна багатоваріантна колоризація зображень на**  
**базі гібридної архітектури U-Net та V-Mamba»**

Виконав:

студент 4 курсу, групи ФІ-21

Климентьєв Максим Андрійович

\_\_\_\_\_

Керівник:

асистент кафедри ММАД д-р філософії

Железняков Дмитро Валентинович

\_\_\_\_\_

Рецензент:

посада, степінь, звання

Прізвище Ім'я По-батькові

\_\_\_\_\_

Засвідчую, що у цій дипломній  
роботі немає запозичень з праць  
інших авторів без відповідних  
посилань.

Студент \_\_\_\_\_

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ  
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ  
імені Ігоря СІКОРСЬКОГО»**

**Навчально-науковий фізико-технічний інститут  
Кафедра математичного моделювання та аналізу даних**

Рівень вищої освіти — перший (бакалаврський)  
Спеціальність — 113 Прикладна математика,  
ОПП «Математичні методи моделювання, розпізнавання образів та  
комп'ютерного зору»

**ЗАТВЕРДЖУЮ**

В.о. завідувача кафедри

\_\_\_\_\_ Іван ТЕРЕЩЕНКО

«\_\_\_» \_\_\_\_\_ 2026 р.

**ЗАВДАННЯ  
на дипломну роботу**

Студент: Климентьєв Максим Андрійович

1. Тема роботи: *«Інтерактивна багатоваріантна колоризація зображень на базі гібридної архітектури U-Net та V-Mamba»*, науковий керівник: асистент кафедри ММАД д-р філософії Железняков Дмитро Валентинович, затверджені наказом по університету №\_\_\_ від «\_\_\_» \_\_\_\_\_ 2026 р.

2. Термін подання студентом роботи: «\_\_\_» \_\_\_\_\_ 2026 р.

3. Об'єкт дослідження: процес інтерактивної багатоваріантної колоризації зображень.

4. Предмет дослідження: моделі глибокого навчання для колоризації зображень, метрики для їх порівняльного аналізу, а також методи суб'єктивного оцінювання якості інтерактивної колоризації.

5. Перелік завдань:

- проаналізувати сучасний стан досліджень у задачі колоризації зображень та виявити обмеження існуючих методів глибокого навчання, зокрема архітектур на базі CNN, GAN, Transformer та Diffusion;

- сформувати набори даних для навчання і тестування, а також розробити алгоритм генерації кольорових підказок для імітації взаємодії з користувачем;

- спроектувати та реалізувати власну модель колоризації на базі гібридної архітектури U-Net та V-Mamba, а також інтегрувати базові моделі інших архітектур для порівняння;

- розробити програмне забезпечення із графічним інтерфейсом для виконання інтерактивної багатоваріантної колоризації;

- здійснити навчання розробленої моделі та провести комплексне оцінювання її ефективності шляхом порівняльного тестування за об'єктивними метриками і проведення користувацького дослідження.

6. Орієнтовний перелік графічного (ілюстративного) матеріалу:  
Презентація доповіді.

7. Дата видачі завдання: 3 вересня 2025 р.

#### Календарний план

| № з/п | Назва етапів виконання дипломної роботи                        | Термін виконання         | Примітка |
|-------|----------------------------------------------------------------|--------------------------|----------|
| 1     | Узгодження теми роботи із науковим керівником                  | 01-15 вересня 2025 р.    | Виконано |
| 2     | Огляд опублікованих джерел за тематикою дослідження            | Вересень-грудень 2025 р. | Виконано |
| 3     | Планування кроків проведення дослідження та збирання датасетів | Квітень 2026 р.          | Виконано |
| 4     | Написання програмного забезпечення та проведення досліджень    | Квітень-травень 2026 р.  | Виконано |
| 5     | Аналіз отриманих результатів                                   | Травень 2026 р.          | Виконано |
| 6     | Оформлення дипломної роботи                                    | Травень 2026 р.          | Виконано |
| 7     | Попередній захист бакалаврської роботи                         | 2 червня 2026 р.         | Виконано |
| 8     | Подача закінченої бакалаврської роботи на кафедру              | 4 червня 2026 р.         | Виконано |
| 9     | Захист бакалаврської роботи                                    | 18 червня 2026 р.        | Виконано |

Студент

\_\_\_\_\_ Максим КЛИМЕНТЬЄВ

Керівник

\_\_\_\_\_ Дмитро ЖЕЛЕЗНЯКОВ

## РЕФЕРАТ

Кваліфікаційна робота містить: ??? стор., ??? рисунки, ??? таблиць, ??? джерел.

У рефераті роботи ви повинні коротко (два-три абзаци) викласти, що саме було зроблено у цій роботі. Перші три речення реферату (після статистичних даних) повинні окреслити мету роботи, об'єкт та предмет дослідження. Після цього викладаються основні результати, одержані в ході дослідження.

Наприкінці анотації великими літерами зазначаються ключові слова. Ось так:

КЛЮЧОВІ СЛОВА, СИМЕТРИЧНА КРИПТОГРАФІЯ, ФІЗТЕХ  
НАЙКРАЩІЙ

## **ABSTRACT**

The English abstract must be the exact translation of the Ukrainian “annotation” (including statistical data and keywords).

## ЗМІСТ

|                                                                                                                  |    |
|------------------------------------------------------------------------------------------------------------------|----|
| Перелік умовних позначень, скорочень і термінів .....                                                            | 8  |
| Вступ .....                                                                                                      | 10 |
| 1 Теоретичні основи колоризації зображень .....                                                                  | 13 |
| 1.1 Поняття та історія колоризації .....                                                                         | 13 |
| 1.2 Підходи до колоризації .....                                                                                 | 15 |
| 1.2.1 Згорткові нейронні мережі .....                                                                            | 15 |
| 1.2.2 Генеративно-змагальні мережі .....                                                                         | 15 |
| 1.2.3 Трансформери .....                                                                                         | 16 |
| 1.2.4 Дифузійні моделі .....                                                                                     | 16 |
| 1.2.5 Селективні моделі простору станів .....                                                                    | 16 |
| 1.3 Інтерактивна колоризація .....                                                                               | 17 |
| 1.4 Метрики якості колоризації .....                                                                             | 17 |
| 1.5 Більш детально про існуючі підходи до колоризації .....                                                      | 19 |
| 1.5.1 Методи на основі втручання користувача .....                                                               | 19 |
| 1.5.2 Методи перенесення кольору зі зразка .....                                                                 | 20 |
| 1.5.3 Згорткові нейронні мережі .....                                                                            | 20 |
| 1.5.4 Генеративно-змагальні мережі .....                                                                         | 21 |
| 1.5.5 Трансформери та механізми уваги .....                                                                      | 21 |
| 1.5.6 Дифузійні моделі .....                                                                                     | 23 |
| 1.5.7 Селективні моделі простору станів та архітектура Mamba .....                                               | 24 |
| 1.6 Порівняння різних методів колоризації .....                                                                  | 25 |
| Висновки до розділу 1 .....                                                                                      | 27 |
| 2 Проєктування та реалізація гібридної архітектури і системи<br>інтерактивної багатоваріантної колоризації ..... | 29 |
| 2.1 Загальна концепція та конвеєр обробки даних .....                                                            | 29 |
| 2.1.1 Стратегія симуляції підказок користувача під час навчання .....                                            | 30 |
| 2.1.2 Автоматизована симуляція підказок на етапі валідації .....                                                 | 32 |
| 2.1.3 Алгоритм стохастичної генерації підказок для багатоваріантності .....                                      | 32 |

|       |                                                                         |         |
|-------|-------------------------------------------------------------------------|---------|
| 2.2   | Компонентна деталізація та алгоритми навчання гібридної моделі ...      | 7<br>34 |
| 2.2.1 | Згорткові блоки енкодера та декодера .....                              | 34      |
| 2.2.2 | Двонаправлене просторове сканування .....                               | 35      |
| 2.2.3 | Механізм спільного використання ваг .....                               | 35      |
| 2.2.4 | Загальна архітектура .....                                              | 36      |
| 2.2.5 | Формулювання функцій втрат .....                                        | 37      |
| 2.3   | Програмна реалізація та графічний інтерфейс користувача .....           | 39      |
|       | Висновки до розділу 2 .....                                             | 40      |
| 3     | Експериментальний аналіз результатів та оцінка ефективності системи ... | 41      |
| 3.1   | Апаратне забезпечення та параметри навчання .....                       | 41      |
| 3.2   | Методологія тестування та базові моделі .....                           | 42      |
| 3.3   | Об'єктивний аналіз якості генерації та швидкодії .....                  | 43      |
| 3.4   | Оцінка перцептивної реалістичності та точності керування .....          | 44      |
| 3.5   | Візуальні результати .....                                              | 45      |
| 3.6   | Користувацьке дослідження якості колоризації .....                      | 45      |
| 3.6.1 | Методологія проведення опитування .....                                 | 46      |
| 3.6.2 | Результати першого етапу: Оцінка реалістичності .....                   | 47      |
| 3.6.3 | Результати другого етапу: Точність слідування підказкам .....           | 48      |
| 3.6.4 | Результати третього етапу: Оцінка багатоваріантності .....              | 49      |
| 3.6.5 | Висновки за результатами користувацького дослідження .....              | 49      |
|       | Висновки до розділу 3 .....                                             | 50      |
|       | Висновки .....                                                          | 51      |
|       | Перелік посилань .....                                                  | 53      |
|       | Додаток А Фрагменти вихідного коду програми .....                       | 58      |
| A.1   | Реалізація двонаправленого блоку Shared Mamba .....                     | 58      |
| A.1.1 | Допоміжні модулі .....                                                  | 58      |
| A.1.2 | Основний модуль .....                                                   | 59      |
| A.2   | Генерація Гауссівських підказок .....                                   | 60      |
| A.3   | Багатоваріантність .....                                                | 62      |
| A.4   | Комбінована функція втрат .....                                         | 65      |

## ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СКОРОЧЕНЬ І ТЕРМІНІВ

### Скорочення

- cGAN — умовна генеративно-змагальна мережа (англ. conditional Generative Adversarial Network)
- CLIP — контрастивне попереднє навчання мови та зображень (англ. Contrastive Language-Image Pre-training)
- CNN — згорткова нейронна мережа (англ. Convolutional Neural Network)
- FID — відстань Фреше (англ. Fréchet Inception Distance)
- GAN — генеративно-змагальна мережа (англ. Generative Adversarial Network)
- KID — ядерна відстань (англ. Kernel Inception Distance)
- LPIPS — перцептивна метрика на базі глибоких ознак (англ. Learned Perceptual Image Patch Similarity)
- MAE — середньоабсолютна похибка (англ. Mean Absolute Error)
- MSE — середньоквадратична похибка (англ. Mean Squared Error)
- PSNR — пікове відношення сигналу до шуму (англ. Peak Signal-to-Noise Ratio)
- SSIM — індекс структурної схожості (англ. Structural Similarity Index)
- SSM — селективна модель простору станів (англ. Selective State Space Model)
- VRAM — відеопам'ять (англ. Video Random Access Memory)

### Терміни

- Autoregressive Transformer — авторегресивний трансформер



|                 |                           |
|-----------------|---------------------------|
| baseline model  | — базова модель           |
| batch size      | — розмір пакету           |
| Diffusion Model | — дифузійна модель        |
| Transformer     | — трансформер             |
| weight decay    | — коефіцієнт згасання ваг |

## ВСТУП

**Актуальність дослідження.** Стрімкий розвиток методів глибокого навчання відкрив нові можливості для розв’язання складних задач генерації та обробки візуальних даних, однією з яких є інтерактивна колоризація монохромних зображень. Актуальність розробки нових підходів у цій сфері зумовлена стабільним практичним попитом на інтелектуальні інструменти розфарбовування у креативних індустріях, а також для задач реставрації архівних фотографій.

Незважаючи на значний прогрес у галузі комп’ютерного зору, існуючі архітектурні рішення мають ряд суттєвих обмежень. Традиційні архітектури на базі згорткової нейронної мережі (англ. Convolutional Neural Network, CNN), умовної генеративно-змагальної мережі (англ. Conditional Generative Adversarial Network, cGAN) та трансформера (англ. Transformer) є детермінованими. Такі моделі зводять задачу до однозначного відображення, що значно ускладнює природну генерацію альтернативних кольорових рішень для одного й того ж вхідного зображення без застосування специфічних модифікацій. З іншого боку, такі генеративні архітектури, як генеративно-змагальна мережа (англ. Generative Adversarial Network, GAN), дифузійна модель (англ. Diffusion Model) та авторегресивний трансформер (англ. Autoregressive Transformer), здатні забезпечувати багатоваріантність генерації. Однак їх значна ресурсоемність, що зумовлена високою обчислювальною складністю та тривалим часом інференсу, робить ці підходи важкозастосовними для миттєвої інтерактивної взаємодії у реальному часі, де від системи вимагається швидкий відгук на кольорові підказки.

У цьому контексті перспективним напрямком є дослідження новітніх архітектур, зокрема селективних моделей простору станів (англ. Selective State Space Models, SSM). На сьогодні рішення на основі SSM для задач інтерактивної багатоваріантної колоризації зображень загального вигляду відсутні, а існуючі поодинокі дослідження обмежені вузькоспеціалізованими доменами даних.

Оскільки чисті архітектури такого класу мають обмежені можливості щодо збереження чітких локальних просторових ознак та текстур зображення, виникає обґрунтована необхідність проектування саме гібридних моделей. Це зумовлює доцільність поєднання глобального контексту SSM із локальними властивостями згорткових підходів для створення обчислювально ефективних архітектур, здатних забезпечити високу якість генерації та миттєвий відгук на дії користувача.

**Мета дослідження.** Розробка моделі колоризації на базі запропонованої гібридної архітектури U-Net + B-Mamba, створення програмного забезпечення інтерактивної колоризації з можливістю вибору різних моделей та проведення порівняльного аналізу із сучасними базовими моделями (англ. baseline models).

**Задача дослідження.** Для досягнення поставленої мети необхідно вирішити такі завдання:

1) проаналізувати сучасний стан досліджень у задачі колоризації зображень та виявити обмеження існуючих методів глибокого навчання, зокрема архітектур на базі CNN, GAN, Transformer та Diffusion;

2) сформувати набори даних для навчання і тестування, а також розробити алгоритм генерації кольорових підказок для імітації взаємодії з користувачем;

3) спроектувати та реалізувати власну модель колоризації на базі гібридної архітектури U-Net та B-Mamba, а також інтегрувати базові моделі інших архітектур для порівняння;

4) розробити програмне забезпечення із графічним інтерфейсом для виконання інтерактивної багатоваріантної колоризації;

5) здійснити навчання розробленої моделі та провести комплексне оцінювання її ефективності шляхом порівняльного тестування за об'єктивними метриками і проведення користувацького дослідження.

*Об'єктом дослідження* є процес інтерактивної багатоваріантної колоризації зображень.

*Предметом дослідження* є моделі глибокого навчання для колоризації зображень, метрики для їх порівняльного аналізу, а також методи суб'єктивного оцінювання якості інтерактивної колоризації.

При розв’язанні поставлених завдань використовувались такі *методи дослідження*: методи глибокого навчання та методи комп’ютерного зору, методи об’єктивного та суб’єктивного оцінювання якості результатів.

**Наукова новизна.** Наукова новизна отриманих результатів полягає у наступному:

- вперше застосовано SSM до задачі інтерактивної багатоваріантної колоризації монохромних зображень загального вигляду, що дозволило розширити сферу використання цього класу моделей за межі аналізу вузькоспеціалізованих даних, а саме інфрачервоних та супутникових знімків;
- запропоновано нову модифіковану гібридну архітектуру, на базі поєднання U-Net та B-Mamba, для задач інтерактивної колоризації. За рахунок інтеграції механізму двонаправленого сканування зі спільними вагами вперше вдалося ефективно об’єднати здатність SSM до захоплення глобального контексту з локальною просторовою точністю згорткових мереж, суттєво оптимізувавши споживання відеопам’яті.

**Практичне значення.** Практичне значення отриманих результатів полягає в тому, що розроблена обчислювально ефективна гібридна архітектура та створений на її основі програмний інструмент можуть бути використані в задачах інтерактивної колоризації зображень у творчих індустріях та реставраційних проєктах. Зокрема:

- завдяки оптимізації використання відеопам’яті, запропонована модель дозволяє виконувати інтерактивну багатоваріантну колоризацію на локальних системах, оснащених CUDA-сумісними графічними прискорювачами, без необхідності залучення серверних потужностей;
- створено програмний прототип із графічним інтерфейсом, що забезпечує можливість інтерактивного керування процесом колоризації за допомогою кольорових підказок у режимі реального часу;
- розроблений модульний пайплайн навчання та метричного оцінювання дозволяє розробникам швидко адаптувати запропоновану архітектуру під нові набори даних або суміжні задачі комп’ютерного зору без необхідності розробки інфраструктури з нуля.

# 1 ТЕОРЕТИЧНІ ОСНОВИ КОЛОРИЗАЦІЇ ЗОБРАЖЕНЬ

У цьому розділі розглядаються основні теоретичні аспекти колоризації зображень, включно з історією розвитку методів, класичними алгоритмічними підходами та сучасними нейромережевими моделями. Особлива увага приділяється різноманітним підходам: від архітектур на основі згорткової нейронної мережі (англ. Convolutional Neural Network, CNN) до дифузійної моделі (англ. Diffusion Model), а також методам оцінки якості отриманих результатів.

Основна мета цього розділу – проаналізувати еволюцію методів колоризації від класичних CNN до сучасних архітектур типу трансформер (англ. Transformer), виявити їхні обмеження щодо обчислювальної складності та збереження просторових контурів об'єктів, і на основі цього обґрунтувати необхідність переходу до гібридних ієрархічних архітектур на базі селективної моделі простору станів (англ. Selective State Space Model, SSM).

## 1.1 Поняття та історія колоризації

Колоризація – це процес додавання кольорової інформації до монохромних зображень. З математичної точки зору, цю задачу можна сформулювати як відображення одноканального сірого зображення у триканальне кольорове зображення, наприклад, у просторі RGB, CIELAB або YUV [1]. Оскільки одному значенню яскравості може відповідати безліч кольорових відтінків, задача є невизначеною та має характер «один до багатьох», що вимагає використання апріорних знань про семантику сцени.

Історично методи колоризації можна поділити на три основні етапи:

- 1) **ручна та напівавтоматична колоризація:** вимагала значних зусиль художників або використання простих евристик на основі сегментації;
- 2) **методи на основі глибокого навчання:** поява великих наборів даних,

таких як ImageNet та COCO, дозволила тренувати CNN для передбачення кольору [2]. Знаковим став підхід *Colorful Image Colorization* [3], де задачу було переформульовано з класичної регресії на класифікацію. Замість того, щоб мінімізувати усереднену похибку, яка призводить до тьмяних кольорів, модель передбачає розподіл ймовірностей для кожного пікселя по палітрі відтінків. Подальший розвиток цього етапу ознаменувався впровадженням генеративно-змагальної мережі (англ. Generative Adversarial Network, GAN), що дозволило подолати проблему розмитості та суттєво підвищити фотореалістичність результатів завдяки змагальному процесу навчання;

**3) генеративні та послідовні моделі:** сучасний етап, що характеризується використанням механізмів глобальної уваги трансформерів та дифузійних моделей для генерації високоякісних текстур, а також впровадженням SSM для лінійного моделювання контексту зображень та забезпечення високої семантичної узгодженості [4, 5].

Розвиток методів глибокого навчання, зокрема поява архітектур U-Net та згаданих раніше GAN, дозволив перейти від простого розфарбовування до генерації фотореалістичних зображень [6]. Останні дослідження все частіше фокусуються на дифузійних моделях, які демонструють передові результати у задачах синтезу зображень [7].

Поряд із цим, активного розвитку набули архітектури на базі механізмів глобальної уваги. Проте висока обчислювальна складність класичних трансформерів, а також обмеження лінійних та інших ефективних апроксимацій у точному моделюванні глобального контексту [8, 9], роблять їх менш придатними для інтерактивних задач.

Водночас новітні архітектури на базі SSM здатні забезпечити глобальне рецептивне поле з лінійною складністю. Хоча вони й почали з'являтися у вузькоспеціалізованих задачах відновлення інфрачервоних [10, 11] та супутникових знімків [12], вони залишаються недослідженими в контексті інтерактивної колоризації фотографій. Відсутність рішень, які б поєднували лінійну обчислювальну ефективність SSM із точним просторовим контролем за допомогою користувацьких підказок, створює простір для наукового пошуку та

підкреслює актуальність запропонованого в цій роботі гібридного підходу.

## 1.2 Підходи до колоризації

Підходи до колоризації базуються на різних архітектурах нейронних мереж. Опустимо гібридні архітектури та розглянемо деякі з базових.

### 1.2.1 Згорткові нейронні мережі

Ранні підходи використовували автоенкодери для прямого передбачення каналів кольоровості (наприклад,  $a$  та  $b$  у просторі CIE Lab).

Ларссон та ін. [13, 3] запропонували використовувати ознаки, вивчені для класифікації об'єктів, для задачі колоризації, демонструючи тісний зв'язок між семантикою об'єкта та його кольором.

Інший популярний метод, *Deep Koalarization* [14], використовує Inception-ResNet-v2 для вилучення ознак та подвійний декодер для відновлення кольору.

### 1.2.2 Генеративно-змагальні мережі

З метою усунення ефекту недостатньої насиченості кольорів були запропоновані різні GAN.

- Модель *Pix2Pix* [6] використовує умовний GAN (англ. Conditional Generative Adversarial Network, cGAN) для перетворення зображень, що дозволяє генерувати більш чіткі та реалістичні результати.

- Метод *ChromaGAN* [15] поєднує геометричну інформацію з семантичним розподілом класів для покращення правдоподібності розфарбовування.

- Також існують підходи для непарного навчання, такі як CycleGAN [16], що дозволяють тренувати моделі без наявності пар «чорно-біле – кольорове».

### 1.2.3 Трансформери

З появою механізму уваги дослідники почали застосовувати трансформери для колоризації.

Наприклад, *Colorization Transformer* [17] та *UniColor* [18] використовують архітектуру трансформерів для моделювання довгих залежностей у зображенні, що покращує узгодженість кольорів на великих відстанях.

Як ще один приклад: *DDColor* [19] використовує подвійні декодери та запити кольору (англ. Color Queries) для досягнення фотореалістичності.

### 1.2.4 Дифузійні моделі

Найбільш перспективним напрямком на сьогодні є дифузійні моделі. Принцип їхньої роботи полягає в ітеративному оберненні процесу деградації зображення для відновлення його структури. Хоча у класичних підходах під деградацією розуміють поступове додавання випадкового гаусового шуму, сучасні методи розширюють це поняття на будь-які типи спотворень.

- *Palette* [7] – універсальний фреймворк для перетворення зображень, який перевершує GAN та регресійні моделі без необхідності налаштування під конкретну задачу.

- *Cold Diffusion* [20] пропонує підхід до інверсії довільних перетворень зображень без використання гаусового шуму, що відкриває нові можливості для відновлення зображень.

### 1.2.5 Селективні моделі простору станів

Новітнім трендом у глибокому навчанні є впровадження SSM, зокрема архітектури Mamba [5]. Вони розроблені як обчислювально ефективна альтернатива архітектурам типу трансформер. Головною перевагою SSM є здатність моделювати наскрізні глобальні залежності із суворо лінійною



складністю  $\mathcal{O}(N)$  відносно довжини послідовності, що є критичним фактором для забезпечення інтерактивності та роботи систем у реальному часі.

### 1.3 Інтерактивна колоризація

Оскільки колоризація є відносно суб'єктивною задачею, важливу роль відіграють методи, що дозволяють користувачеві впливати на результат:

1) **колоризація на основі прикладів:** методи, такі як *Deep Exemplar-based Colorization* [21], використовують референсне кольорове зображення для перенесення колірної палітри на цільове чорно-біле;

2) **колоризація на основі тексту:** з розвитком мультимодальних моделей, таких як CLIP, з'явилася можливість керувати кольором за допомогою текстових описів, зокрема *L-CAD* [22] використовує дифузійні пріори для колоризації на основі описів будь-якого рівня деталізації, а *Diffusing Colors* [23] пропонує фреймворк, де текстові підказки керують процесом дифузії, забезпечуючи баланс між автоматизацією та контролем;

3) **колоризація на основі підказок користувача:** методи *Real-Time User-Guided Image Colorization* [24] та *iColoriT* [25] дозволяють користувачеві ставити кольорові точки або штрихи, які мережа поширює на відповідні семантичні регіони.

Сучасна тенденція, як показано в *Control Color* [26] та *UniColor* [18], полягає у створенні уніфікованих фреймворків, що підтримують різні типи умов, такі як текст, зразок, точки, штрихи, в одній моделі.

### 1.4 Метрики якості колоризації

Оцінка якості колоризації є нетривіальним завданням через фундаментальну неоднозначність розв'язку: одному чорно-білому вхідному сигналу відповідає ціла множина семантично та візуально коректних колірних варіацій. Оскільки відображення має характер  $1 \rightarrow N$ , використання

лише одного критерію не дозволяє повноцінно оцінити роботу моделі. Для комплексного аналізу розроблених архітектур у роботі застосовується комбінований набір метрик, що охоплює такі аспекти:

- **метрики локальної та глобальної похибки:** у задачах інтерактивної колоризації похибку доцільно розділяти за областю обчислення. Зокрема, метрика середньоквадратичної похибки (англ. Mean Squared Error, MSE) та середньоабсолютної похибки (англ. Mean Absolute Error, MAE) оцінюють загальне відхилення кольору на всьому зображенні, тоді як спеціалізовані метрики  $MSE_{HINT}$  та  $MAE_{HINT}$  розраховують виключно в координатах наданих користувачем підказок. Наближення  $MSE_{HINT}$  та  $MAE_{HINT}$  до нуля є індикатором того, що мережа не ігнорує користувацький ввід, а ідеально інтегрує його у фінальний результат;

- **піксельні та структурні метрики:** пікове відношення сигналу до шуму (англ. Peak Signal-to-Noise Ratio, PSNR) та індекс структурної схожості (англ. Structural Similarity Index, SSIM) – класичні методи порівняння, які вимірюють пряме відхилення згенерованого зображення від еталонного кольорового зображення. Хоча вони зручні для математичного аналізу, у задачах колоризації їхня репрезентативність є обмеженою: якщо модель згенерує семантично правильну червону машину замість оригінальної синьої, ці метрики зафіксують значну помилку, незважаючи на високу візуальну правдоподібність результату;

- **перцептивні генеративні метрики:** перцептивна метрика на базі глибоких ознак (англ. Learned Perceptual Image Patch Similarity, LPIPS) використовується для оцінки попарної схожості зображень на рівні глибоких ознак нейронної мережі VGG. Натомість відстань Фреше (англ. Fréchet Inception Distance, FID) та ядерна відстань (англ. Kernel Inception Distance, KID) оцінюють відстань між розподілами ознак усього набору даних, використовуючи при цьому Inception. Зниження цих показників свідчить про те, що статистика згенерованих зображень наближається до реальних фотографій. Оскільки KID обчислюється ітеративно на випадкових підвибірках, результат подається у форматі середнього значення  $KID_{mean}$  та стандартного відхилення  $KID_{std}$ , що характеризує стабільність моделі;

- **обчислювальна та апаратна ефективність:** оскільки розроблювана архітектура орієнтована на інтерактивність, критичними параметрами є час інференсу, Time, та пікове споживання відеопам'яті (англ. Video Random Access Memory, VRAM).

## 1.5 Більш детально про існуючі підходи до колоризації

До епохи глибокого навчання колоризація розглядалася переважно як задача оптимізації. Класичні алгоритмічні методи можна розділити на дві великі групи: методи на основі втручання користувача та методи перенесення кольору зі зразка.

### 1.5.1 Методи на основі втручання користувача

Цей підхід базується на припущенні, що сусідні пікселі з подібною інтенсивністю повинні мати подібний колір. Користувач наносить на чорно-біле зображення кольорові штрихи, які слугують граничними умовами. Задача зводиться до мінімізації цільової функції енергії  $J$  [1, 27]:

$$J(U) = \sum_r \left( U(r) - \sum_{s \in N(r)} w_{rs} U(s) \right)^2,$$

де  $U(r)$  – колір у пікселі  $r$ ,  $N(r)$  – окіл пікселя, а  $w_{rs}$  – ваговий коефіцієнт, що залежить від подібності яскравості пікселів  $r$  та  $s$ .

Перевагою методу є повний контроль користувача, проте він вимагає значних ручних зусиль і часто дає артефакти на межах об'єктів зі слабким контрастом.

### 1.5.2 Методи перенесення кольору зі зразка

У цьому підході колір переноситься з референсного кольорового зображення на цільове чорно-біле шляхом зіставлення статистик яскравості та текстур.

Ранні роботи [28, 29] використовували зіставлення гістограм у декорельованому колірному просторі  $l\alpha\beta$  (простір Рудермана), що дозволяло маніпулювати колірними каналами незалежно один від одного без виникнення візуальних артефактів.

Хоча цей метод є більш автоматизованим, він критично залежить від вдалого вибору референсного зображення та часто призводить до помилкового забарвлення семантично різних об'єктів, що мають схожу текстуру.

### 1.5.3 Згорткові нейронні мережі

Поява CNN дозволила моделювати складні залежності між формою об'єкта та його ймовірним кольором, використовуючи великі набори даних.

Перші нейромережеві методи намагалися прямо передбачити значення каналів  $ab$  у просторі Lab, мінімізуючи MSE:

$$\mathcal{L}_{L2} = \frac{1}{2} \sum_{h,w} \|Y_{gt} - \hat{Y}\|_2^2,$$

де  $Y_{gt}$  – справжнє зображення, а  $\hat{Y}$  – згенероване.

Проте, як зазначають Zhang et al. [3], використання MSE призводить до усереднення всіх можливих кольорів, що дає ненасичені результати.

Для вирішення цієї проблеми задачу було переформульовано як мультимодальну класифікацію, де мережа передбачає розподіл ймовірностей для квантованих відтінків кольору, що значно підвищило яскравість результатів.

### 1.5.4 Генеративно-змагальні мережі

Мережі GAN, зокрема архітектура Pix2Pix [6], здійснили прорив у чіткості зображень. Генератор  $G$  намагається створити реалістичне зображення, щоб обдурити дискримінатор  $D$ , який вчиться відрізняти справжні кольорові фото від згенерованих. Функція втрат cGAN має вигляд:

$$\mathcal{L}_{cGAN}(G, D) = \mathbb{E}_{x,y}[\log D(x, y)] + \mathbb{E}_{x,z}[\log(1 - D(x, G(x, z)))],$$

де  $x$  – вхідне чорно-біле зображення,  $y$  – цільове кольорове зображення, а  $z$  – вектор випадкового шуму.

Підходи на кшталт ChromaGAN [15] інтегрують семантичну інформацію у процес генерації, що дозволяє уникнути грубих помилок, наприклад, зеленого неба.

### 1.5.5 Трансформери та механізми уваги

Останні дослідження [17, 18, 19] показують, що класичні CNN мають локальне рецептивне поле, через що вони погано справляються з моделюванням глобальних зв'язків. Це часто призводить до семантичних помилок: наприклад, різні частини одного об'єкта, перекриті іншими елементами сцени, можуть бути розфарбовані в різні кольори.

Перехід до архітектур на базі трансформерів дозволив вирішити цю проблему. Вони використовують механізм глобальної уваги, який дозволяє кожному патчу зображення взаємодіяти з усіма іншими патчами одночасно, враховуючи контекст усієї сцени. Це є критично важливим для просторово узгодженого забарвлення великих областей та підтримки єдиної логіки освітлення.

Зокрема, модель Colorization Transformer [17] вперше успішно застосувала цей підхід, розбивши зображення на послідовність патчів і розглядаючи задачу колоризації як задачу прогнозування послідовності токенів, подібно до генерації

тексту. Модель генерує кольори авторегресійно, що забезпечує високу якість, проте робить процес інференсу надзвичайно повільним.

Значним кроком у напрямку інтерактивності стала модель UniColor [18]. Вона запропонувала уніфікований фреймворк, який здатний обробляти мультимодальні умови – від автоматичної колоризації до генерації на основі текстових описів, референсних зображень або точкових підказок. Використання трансформера дозволило ефективно інтегрувати ці різномірні підказки у єдиний простір репрезентацій.

Для вирішення проблеми розмиття контурів, яка часто виникає у чистих трансформерів, архітектура DDColor [19] впровадила систему подвійних декодерів. Один декодер відповідає виключно за відновлення просторової роздільної здатності, тобто за піксельний рівень, а інший – за формування кольорових семантичних ознак, тобто за глобальний контекст, після чого їхні результати зливаються. Це дозволило досягти високого рівня фотореалістичності.

Проте, незважаючи на беззаперечні переваги в якості генерації, використання класичного механізму уваги накладає жорсткі апаратні обмеження. Обчислювальна складність глобальної уваги зростає квадратично  $\mathcal{O}(N^2)$  зі збільшенням роздільної здатності зображення. Це робить такі моделі вкрай вимогливими до обсягу (VRAM) і ускладнює їх використання для інтерактивної колоризації зображень у високій якості в реальному часі.

Спроби подолати цю проблему за рахунок так званих «ефективних трансформерів» призводять до нових компромісів. Наприклад, обчислення уваги в локальних вікнах, як-от, в Swin Transformer [30], або використання математичних лінійних та інших апроксимацій можуть обмежувати здатність моделі до повного глобального моделювання контексту [8, 9]. У задачі інтерактивної колоризації це стає критичним недоліком, оскільки нейромережа втрачає здатність швидко та безшовно поширювати колір від точкової підказки користувача на віддалені ділянки об'єкта.

### 1.5.6 Дифузійні моделі

На відміну від GAN, які генерують результат за один прохід, дифузійні моделі формують зображення ітеративно. Математична основа класичних дифузійних моделей базується на двох марковських процесах: прямому, який поступово руйнує дані шляхом додавання гаусового шуму, та зворотному, де нейронна мережа вчиться крок за кроком відновлювати оригінальний сигнал, керуючись вхідним чорно-білим зображенням як просторовою умовою.

Цей клас моделей здійснив революцію в синтезі зображень завдяки високій стабільності навчання, тобто відсутності проблеми колапсу мод, який характерний для GAN, та неперевершеній якості генерації дрібних текстур. Знаковою роботою в цій галузі став універсальний фреймворк Palette [7], який продемонстрував, що умовна дифузійна модель здатна перевершити спеціалізовані GAN-архітектури у задачі колоризації без додаткового налаштування під специфіку предметної області.

Поряд із класичним підходом активно розвиваються альтернативні формулювання генеративного процесу. Наприклад, метод Cold Diffusion [20] довів можливість інверсії довільних детермінованих перетворень зображень без використання чистого гаусового шуму, що розширює теоретичні межі застосування дифузійних алгоритмів.

Проте фундаментальним недоліком дифузійних моделей залишається їхня надзвичайно висока обчислювальна вартість на етапі інференсу. Оскільки для створення фінального зображення модель повинна виконати десятки або сотні послідовних ітерацій денойзингу, час генерації зростає на порядки порівняно з моделями прямого проходу. У контексті керованої інтерактивної колоризації, де фундаментальною вимогою є миттєвий візуальний зворотний зв'язок на кожну нову підказку користувача, використання чистих дифузійних моделей є недоцільним через руйнування користувацького досвіду.

### 1.5.7 Селективні моделі простору станів та архітектура Mamba

Новітнім класом нейромережових архітектур, що дозволяє подолати квадратичну складність трансформерів та повільність дифузійних моделей, є SSM, зокрема архітектура Mamba [5]. Вони здатні моделювати наскрізні глобальні контекстні зв'язки з суворю лінійною обчислювальною складністю  $\mathcal{O}(L)$ , що робить їх ідеальними для систем реального часу.

Математично базова система SSM відображає вхідний одновимірний сигнал  $x(t) \in \mathbb{R}$  у прихований стан  $h(t) \in \mathbb{R}^N$  через лінійні диференціальні рівняння:

$$\begin{aligned} h'(t) &= A \cdot h(t) + B \cdot x(t), \\ y(t) &= C \cdot h(t) + D \cdot x(t), \end{aligned}$$

де  $A, B, C, D$  – матриці параметрів системи. При цьому матриця  $A$  характеризує внутрішню динаміку системи та керує еволюцією прихованого стану, тобто відповідає за довготривалу пам'ять, матриця  $D$  описує прямий зв'язок між входом і виходом, виконуючи роль класичного пропускового з'єднання, а матриці  $B$  та  $C$  визначають коефіцієнти взаємодії прихованого стану із вхідним та вихідним сигналами відповідно.

Для обробки дискретних послідовностей безперервні параметри трансформують у дискретні  $(\bar{A}, \bar{B})$  із використанням кроку дискретизації  $\Delta$ . Зазвичай для цього застосовують метод фіксації нульового порядку (англ. Zero-Order Hold):

$$\begin{aligned} \bar{A} &= \exp(\Delta A), \\ \bar{B} &= (\Delta A)^{-1}(\exp(\Delta A) - I) \cdot \Delta B, \end{aligned}$$

що дозволяє переписати систему у дискретному вигляді:  $h_k = \bar{A} \cdot h_{k-1} + \bar{B} \cdot x_k$ ,  $y_k = C \cdot h_k + D \cdot x_k$ .

Головна інновація архітектури Mamba полягає у селективності її



внутрішніх механізмів: параметри  $B$ ,  $C$  та крок дискретизації  $\Delta$  стають динамічними функціями від вхідного сигналу, на відміну від статичних матриць  $A$  та  $D$ . Завдяки такій дискретизації параметрів та використанню апаратно-оптимізованого алгоритму паралельного сканування в пам'яті графічного процесора, модель динамічно фільтрує нерелевантну інформацію та ефективно обробляє довгі послідовності без обчислювальних обмежень механізму уваги.

Однак пряме застосування оригінальної одновимірної Mamba до двовимірних зображень призводить до проблеми втрати локального контексту, оскільки розгортання матриці пікселів у вектор розриває їхню просторову суміжність по вертикалі. Для вирішення цієї проблеми візуальні адаптації на кшталт Vision Mamba [31] та новітні моделі спектральної трансляції [10] використовують механізми двонаправленого або перехресного сканування. Це дозволяє моделі агрегувати ознаки з різних напрямків, повністю відновлюючи просторове розуміння сцени.

У контексті керованої колоризації використання Mamba відкриває унікальну можливість: здатність моделі миттєво та узгоджено поширювати колірну інформацію від точкової підказки користувача на весь об'єкт завдяки глобальному рецептивному полю, уникаючи при цьому експоненційного зростання споживання відеопам'яті. Саме цей синергетичний баланс обчислювальної ефективності та просторової експресивності робить SSM найбільш перспективним фундаментом для розробки сучасних інтерактивних систем розфарбовування.

## 1.6 Порівняння різних методів колоризації

Комплексне порівняння розглянутих методів (табл. 1.1) базується на чотирьох ключових критеріях (Рис. 1.1).

Проведений аналіз демонструє наявність компромісу між обчислювальною лінійністю та збереженням локальних просторових контурів у сучасних послідовних моделях. Чисті трансформери забезпечують високу якість зв'язності, але стають апаратно непридатними для обробки великих зображень

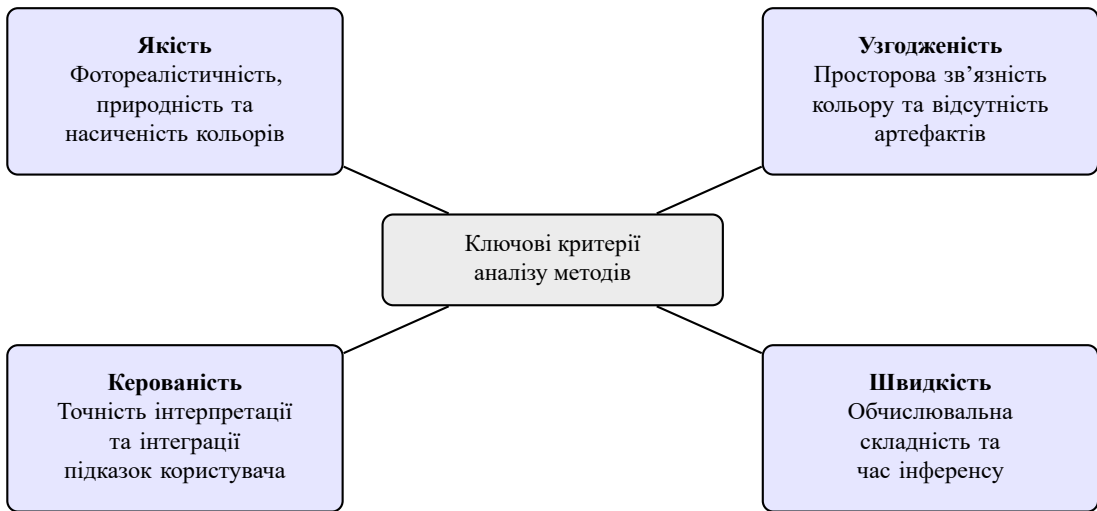


Рисунок 1.1 – Ключові критерії порівняння методів колоризації

Таблиця 1.1 – Порівняльний аналіз методів колоризації зображень

| Метод / Архітектура    | Представники                              | Переваги                                                                                          | Недоліки                                                                                                       |
|------------------------|-------------------------------------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Класичні оптимізаційні | Levin et al. [27], Welsh [29]             | Точний локальний контроль користувача, відсутність семантичних «галюцинацій»                      | Обчислювально повільні, вимагають значної ручної розмітки, пасують перед складними текстурами                  |
| CNN                    | Zhang et al. [3], Larsson [13]            | Висока швидкість інференсу, стабільність процесу навчання                                         | Обмежене рецептивне поле; схильність до генерації усереднених, тьмяних відтінків при мінімізації MSE           |
| GAN                    | Pix2Pix [6], ChromaGAN [15]               | Висока чіткість контурів, насичені та візуально привабливі кольори                                | Нестабільність змагального навчання, схильність до появи кольірних артефактів та плям                          |
| Transformers           | ColTran [17], UniColor [18], DDColor [19] | Моделювання глобального контексту всієї сцени, гнучка мультимодальна керованість                  | Квадратична обчислювальна складність $\mathcal{O}(N^2)$ , критично високі вимоги до обсягу відеопам'яті (VRAM) |
| Diffusion Models       | Palette [7], Cold Diffusion [20]          | Найвища якість генерації (SOTA), бездоганний фотореалізм текстур                                  | Екстремально низька швидкість інференсу через сотні послідовних ітерацій денойзингу                            |
| Mamba SSM              | Vision Mamba [31], ColorMamba [10]        | Суворо лінійна складність $\mathcal{O}(N)$ , стабільне споживання VRAM, глобальне рецептивне поле | Чутливість до одновимірного розгортання 2D-даних, ризик втрати або розмиття локальних контурів об'єктів        |

у реальному часі, тоді як SSM пропонують бажану лінійну ефективність  $\mathcal{O}(N)$ , але потребують додаткових механізмів локального посилення контурів.

Таким чином, гібридні підходи, що поєднують генеративну силу та

швидкість селективних лінійних моделей простору станів із точністю локального вилучення ознак за допомогою ієрархічних згорткових декодерів, виглядають найбільш обґрунтованими та перспективними для вирішення задачі керованої інтерактивної колоризації. Саме розробці та дослідженню такої гібридної архітектури присвячено наступні розділи цієї роботи.

## Висновки до розділу 1

У цьому розділі було проведено огляд теоретичних основ та сучасних методів колоризації зображень. Було показано, що еволюція методів пройшла шлях від простих евристик до складних генеративних моделей, таких як GAN, Transformers, дифузійні фреймворки та селективні моделі простору станів.

Попри значні успіхи безумовної колоризації у досягненні реалістичності результатів, залишаються відкритими питання обчислювальної складності моделей під час роботи із зображеннями високої роздільної здатності, керованості процесом колоризації та забезпечення багатоваріантності результатів. Зокрема, гібридні підходи, мультимодальні підходи та уніфіковані дифузійні й рекурентні структури є найбільш актуальним напрямком досліджень.

Порівняння нейромережових підходів показало, що хоча архітектури GAN забезпечують швидкий інференс, вони часто страждають від нестабільності навчання та появи кольорних артефактів. Дифузійні моделі дозволяють досягти найвищої якості та фотореалізму текстур, проте їхнє практичне застосування жорстко обмежене тривалим часом інференсу через ітеративний характер процесу генерації. У свою чергу, підходи на базі трансформерів ефективно моделюють глобальний контекст усієї сцени, однак квадратичне зростання обчислювальних витрат  $\mathcal{O}(N^2)$  суттєво ускладнює їх масштабування для зображень високої роздільної здатності.

У наступному розділі буде запропоновано власний підхід до вирішення задачі колоризації зображень, що базується на гібридизації архітектури U-Net та Mamba SSM з використанням спільних ваг. Це дозволить ефективно поєднати вилучення низькорівневих просторових ознак, для боротьби з color bleeding,

із лінійним моделюванням глобального контексту та суттєвим збереженням пам'яті графічного процесора.

## 2 ПРОЄКТУВАННЯ ТА РЕАЛІЗАЦІЯ ГІБРИДНОЇ АРХІТЕКТУРИ І СИСТЕМИ ІНТЕРАКТИВНОЇ БАГАТОВАРІАНТНОЇ КОЛОРИЗАЦІЇ

У цьому розділі наведено опис практичної реалізації системи інтерактивної багатоваріантної колоризації зображень та її основної моделі на базі гібридної архітектури U-Net та V-Mamba.

Розглянуто процеси попередньої обробки вхідних даних та алгоритми симуляції користувацьких підказок.

Деталізовано внутрішню будову розробленої нейромережі, наведено обґрунтування двонаправленого сканування послідовностей ознак та описано обрану функцію втрат для забезпечення стабільного навчання моделі.

### 2.1 Загальна концепція та конвеєр обробки даних

Розроблена система реалізує метод інтерактивної колоризації, за якого нейромережа розфарбовує монохромне зображення, спираючись як на власне розуміння семантики об'єктів, так і на точкові колірні підказки користувача.

Математично вхідний потік даних формується шляхом поєднання двох складових:

- канал яскравості  $I_L \in \mathbb{R}^{1 \times H \times W}$ , який представляє компонент  $L$  кольорного простору CIELAB, нормалізований до діапазону  $[-1, 1]$ ;
- триканальний тензор підказок  $I_{\text{hints}} \in \mathbb{R}^{3 \times H \times W}$ , який містить хроматичні значення та інформацію про місцезнаходження кольорових підказок.

Структурно тензор  $I_{\text{hints}}$  складається з двох каналів кольоровості  $a$  та  $b$  розмірністю  $2 \times H \times W$  (нормалізованих у  $[-1, 1]$ ) та маски інтенсивності підказок  $M \in [0, 1]^{1 \times H \times W}$ .

Маска  $M$  необхідна для того, щоб мережа могла відрізнити зони з явними підказками від порожнього простору. Без цього каналу модель сприймала б нульові значення у вільних зонах як вимогу розфарбувати ту частину зображення

у нейтральні ахроматичні кольори.

Перед подачею на вхід енкодера канал яскравості конкатенується з тензором підказок за виміром каналів, утворюючи загальний чотириканальний вхідний масив:

$$X_{\text{in}} = \text{Concat}(I_L, I_{\text{hints}}) \in \mathbb{R}^{4 \times H \times W}.$$

### 2.1.1 Стратегія симуляції підказок користувача під час навчання

Оскільки під час реального використання кількість, розмір та розташування точок є випадковими, процес навчання моделі вимагає динамічної симуляції дій користувача на кожній ітерації. Навчання на фіксованих шаблонах призвело б до перенавчання моделі та ігнорування підказок на інференсі.

Конвеєр підготовки даних під час тренування реалізує таку стохастичну процедуру для кожного батчу:

1) з оригінального кольорового зображення вилучаються істинні канали кольоровості  $Y \in \mathbb{R}^{2 \times H \times W}$ ;

2) випадковим чином визначається цільова кількість підказок  $K$  з дискретного геометричного розподілу  $K \sim \text{Geom}(p = 0.2)$ , в якому математичне сподівання становить  $\mathbb{E}[K] = \frac{1-p}{p}$ , тобто в середньому генерується 4 підказки. Ймовірність появи конкретної кількості підказок  $k$  обчислюється як  $P(K = k) = p(1 - p)^k$ . Зокрема, ймовірність генерації зображення взагалі без підказок, тобто  $K = 0$ , становить 20%, що змушує модель також вчитися безумовній автоматичній колоризації;

3) для імітації фокусу людської уваги координати підказок генеруються за допомогою алгоритму виділення візуальної значущості. Алгоритм обчислює двовимірну функцію щільності ймовірності (PDF) на основі каналу яскравості  $I_L$ .

Спочатку за допомогою згортки з операторами Собеля обчислюється магнітуда градієнтів:

$$G = \sqrt{(I_L * S_x)^2 + (I_L * S_y)^2} + \epsilon,$$

де  $I_L$  – матриця каналу яскравості зображення;  $S_x, S_y$  – ядра оператора Собеля для виділення горизонтальних та вертикальних перепадів яскравості;  $*$  – операція двовимірної згортки;  $\epsilon$  – мала константа, наприклад,  $10^{-6}$ , для уникнення обчислення кореня з нуля та забезпечення чисельної стабільності. Для поширення значущості від контурів усередину об’єктів до матриці  $G$  застосовується просторове усереднення. Отримана карта нормалізується та змішується з рівномірним розподілом  $U$  з ваговим коефіцієнтом  $\alpha = 0.15$ , щоб забезпечити ненульову ймовірність вибору точок навіть в абсолютно однорідних областях зображення:

$$\text{PDF} = (1 - \alpha) \frac{G_{\text{blur}}}{\sum G_{\text{blur}}} + \alpha U,$$

де  $G_{\text{blur}}$  – матриця градієнтів  $G$  після застосування операції просторового згладжування;  $\sum G_{\text{blur}}$  – сума всіх елементів згладженої матриці, використовується як нормалізуючий дільник;  $U$  – матриця рівномірного розподілу ймовірностей, де кожен елемент дорівнює  $\frac{1}{H \cdot W}$ ;  $\alpha$  – ваговий коефіцієнт змішування, який наразі  $\alpha = 0.15$ .

Координати семплюються з цього розподілу без повернення. Додатково застосовується евристика мінімальної просторової відстані, яка відкидає нові координати, якщо вони знаходяться занадто близько до вже існуючих, уникаючи неприродного скупчення кліків;

4) формується маска інтенсивності  $M$ : у вибраних координатах розміщуються двовимірні радіальні гаусові ядра випадкового радіуса та випадкової амплітуди інтенсивності (від 0.2 до 1.0). Це імітує різні розміри пензля користувача та ступінь «впевненості» підказки. Якщо гаусові плями перекриваються, маска оновлюється за принципом максимуму  $M_{\text{new}} = \max(M_{\text{old}}, G_{\text{patch}})$ ;

5) формуються канали підказок  $a$  та  $b$  шляхом поелементного множення істинних значень  $Y$  на згенеровану маску  $M$ .

### 2.1.2 Автоматизована симуляція підказок на етапі валідації

Для об'єктивної оцінки якості моделі під час валідації та тестування, алгоритм перемикається з випадкової генерації у детермінований режим. Алгоритм автоматично знаходить найбільш інформативні зони для колоризації, діючи за такою логікою:

- 1) обчислюється матриця спектральної магнітуди, насиченості кольору, для всього зображення як сума квадратів значень каналів  $a$  та  $b$ ;
- 2) визначаються координати пікселя з максимальною магнітудою, і в цій точці розміщується гаусова підказка фіксованого розміру;
- 3) щоб запобігти вибору точок в одному й тому самому об'єкті, застосовується механізм просторового пригнічення: значення магнітуди в заданому радіусі навколо обраної точки примусово встановлюються у від'ємні значення, а саме  $-1.0$ ;
- 4) процес ітеративно повторюється для заданої кількості підказок, наприклад,  $K = 3$ .

Такий конвеєр валідації гарантує, що модель отримує підказки саме в найскладніших, найбільш яскравих ділянках зображення, що дозволяє коректно оцінити її здатність до поширення кольору на великі просторові відстані.

### 2.1.3 Алгоритм стохастичної генерації підказок для багатоваріантності

З огляду на детерміновану природу базової архітектури, механізм багатоваріантної колоризації винесено на етап попередньої обробки даних. Отримання множини альтернативних результатів для одного зображення забезпечується алгоритмом автоматичного доповнення підказок, який складається з таких етапів:

- 1) на основі користувацького тензора підказок формується маска зайнятості. Усі пікселі, де маска інтенсивності  $M > 0.05$ , позначаються



як пріоритетні та захищаються від автоматичної модифікації, що гарантує збереження намірів користувача.

2) для вибору координат нових підказок алгоритм стохастично, з рівною ймовірністю, обирає одну з двох стратегій семплювання:

- *семплювання на основі градієнтної значущості*, що діє аналогічно до процесу навчання, описаного у п. 2.1.1, та спрямоване на дрібні високочастотні деталі;

- *семплювання семантичних центрів*, де до каналу  $I_L$  застосовується детектор границь Canny, контури інвертуються, після чого обчислюється карта відстаней  $\mathcal{D}$ . Області, які вже містять підказки, пригнічуються за допомогою алгоритму зв'язної заливки із порогом толерантності  $\tau = 20$ . Координати обираються як глобальні максимуми карти  $\mathcal{D}$ , що гарантує точне потрапляння підказок у геометричні центри порожніх об'єктів.

3) для кожної знайденої координати обчислюється випадковий колір. Для забезпечення візуальної природності та запобігання надмірній кислотності результатів, хроматичні значення беруться з нормального розподілу із нульовим математичним сподіванням та обмеженим стандартним відхиленням  $\sigma = 0.25$ , після чого обрізаються до робочого діапазону  $[-1, 1]$ :

$$C_{\text{rand}} = \text{Clamp}(\mathcal{N}(0, \sigma^2), -1, 1).$$

4) нові підказки формуються у вигляді двовимірних гаусових ядер випадкового радіуса від 2 до 8 пікселів та випадкової амплітуди. Згенеровані маски та кольорові значення поєднуються з оригінальним тензором  $I_{\text{hints}}$  за допомогою операції попіксельного максимуму для масок та попіксельного додавання для колірних каналів, утворюючи модифікований вхідний масив для мережі.

Отже, інтеграція стохастичного семплювання до конвеєра даних уможливорює генерацію широкого спектра альтернативних рішень, гарантуючи при цьому абсолютне збереження тих кольорових умов, які були явно задані користувачем.

## 2.2 Компонентна деталізація та алгоритми навчання гібридної моделі

Розроблена гібридна модель базується на ієрархічній архітектурі типу U-Net, у якій класичний центральний згортковий вузол замінено на Mamba.

Такий підхід дозволяє делегувати вилучення локальних просторових ознак та збереження контурів перевіреним згортковим шарам, тоді як завдання глобального поширення кольору від точкових підказок виконується SSM з лінійною обчислювальною складністю.

### 2.2.1 Згорткові блоки енкодера та декодера

Для вилучення просторових ознак та поступового зниження роздільної здатності використовується згортковий енкодер. Кожен його рівень побудований за допомогою *DoubleConv*, структурну схему якої зображено на рис. 2.1.

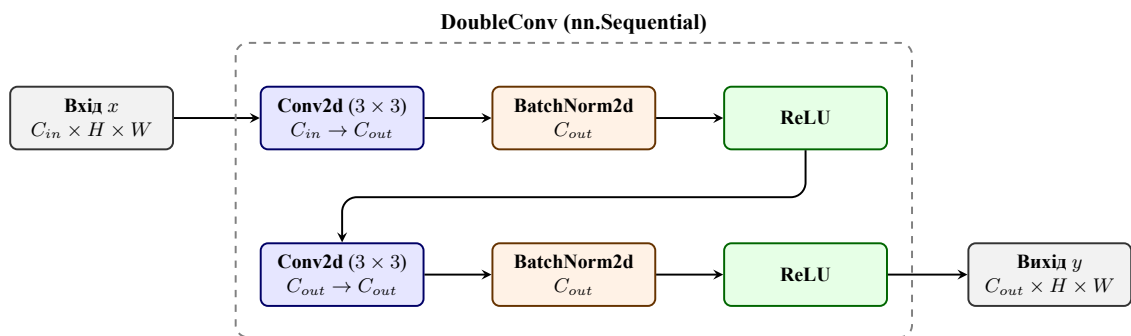


Рисунок 2.1 – Архітектура блоку DoubleConv для енкодера та декодера

Блок складається з двох послідовних згорткових шарів ( $3 \times 3$ ), за кожним з яких слідує нормалізація батчів та нелінійна функція активації ReLU. Після кожного блоку в енкодері застосовується операція просторового стиснення – Max Pooling  $2 \times 2$ , яка зменшує розмірність карт ознак удвічі.

Декодер працює симетрично, використовуючи білінійну інтерполяцію для масштабування.

Критичним елементом архітектури є наскрізні з'єднання, які прокидають карти ознак з енкодера безпосередньо у відповідні шари декодера, що запобігає втраті дрібних контурів об'єктів, тобто розтіканню кольору.

### 2.2.2 Двонаправлене просторове сканування

Оригінальна архітектура Mamba розроблена для обробки одновимірних послідовностей, наприклад, тексту, і сканує дані лише в одному напрямку. Для двовимірних зображень це створює проблему просторової асиметрії: пікселі не отримують контекстної інформації від тих областей, що знаходяться нижче або правіше від них по сітці сканування.

Для вирішення цієї проблеми у центральному згортковому вузлі реалізовано алгоритм двонаправленого сканування. Розгорнута у вектор послідовність просторових ознак  $X \in \mathbb{R}^{B \times L \times D}$ , де  $L$  – це  $H$  помножена на  $W$  та поділена на певний коефіцієнт, який залежить від U-Net, а  $C$  – кількість початкових каналів, у нашому випадку це 4, обробляється паралельно: у природному напрямку та в інвертованому.

Інверсія досягається за допомогою безвитратної операції перегортання тензора у пам'яті (`torch.flip`). Фінальний вихід  $Y$  обчислюється як середнє арифметичне двох проходів:

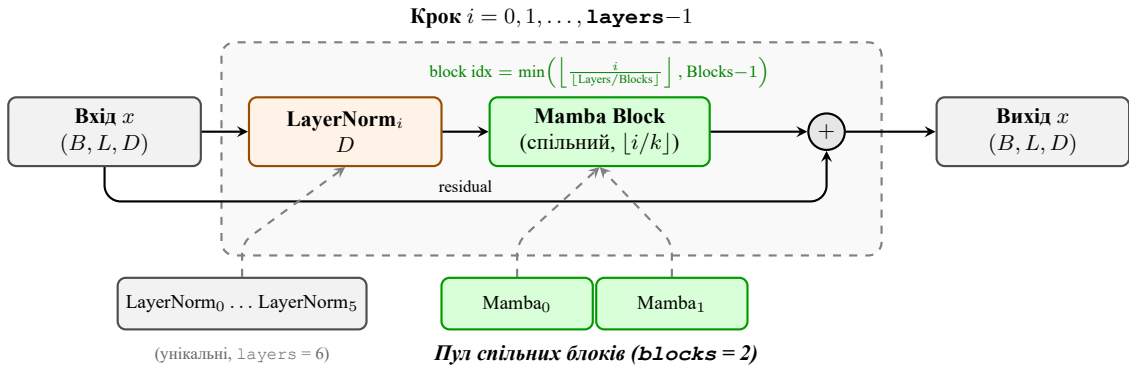
$$Y = \frac{1}{2} (\text{Mamba}_{\text{fwd}}(X) + \text{Flip}(\text{Mamba}_{\text{rev}}(\text{Flip}(X)))) .$$

Це забезпечує кожному просторовому токenu доступ до повного глобального контексту сцени з усіх напрямків.

### 2.2.3 Механізм спільного використання ваг

Використання глибоких послідовностей Mamba-блоків вимагає значного обсягу відеопам'яті графічного процесора. Для оптимізації апаратного споживання без втрати генеративної здатності мережі імплементовано механізм

спільного використання ваг, структурну логіку якого зображено на рис. 2.2.



**Рисунок 2.2** – Внутрішня архітектура та механізм розділення ваг у блоці Shared Mamba

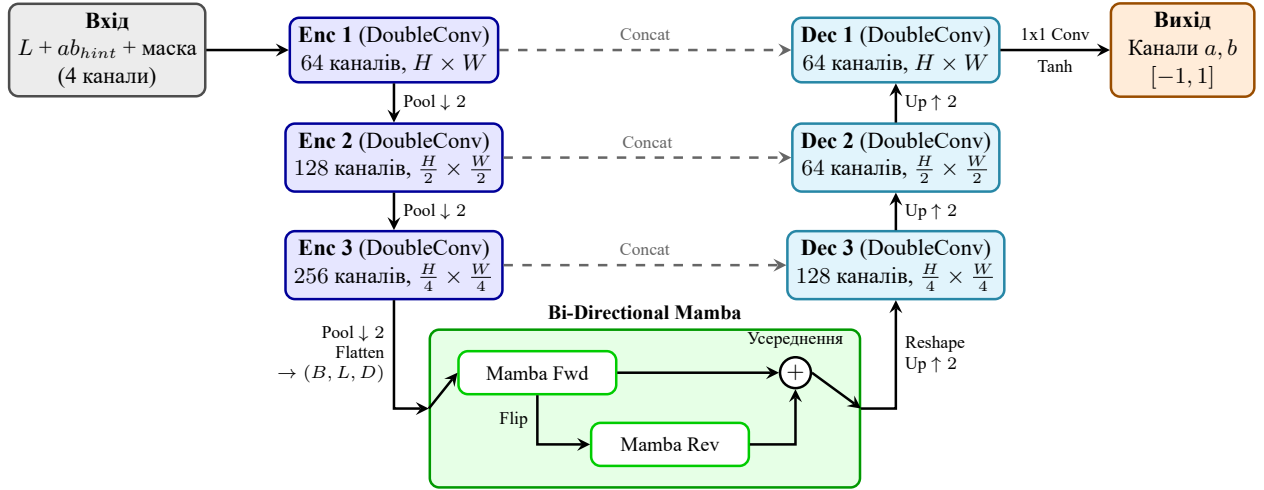
Замість ініціалізації  $L$  унікальних блоків Mamba, модель створює лише обмежений фізичний пул з  $K$  блоків (де  $K < L$ ). Під час прямого проходу шари послідовно перевикористовують ці фізичні блоки згідно з математичним індексом:

$$\text{idx} = \min\left(\left\lfloor \frac{i}{\lfloor L/K \rfloor} \right\rfloor, K-1\right), \quad \text{де } i \in [0, L-1].$$

Водночас, шари пакетної нормалізації (LayerNorm), які передують кожному блоку, залишаються унікальними для кожного кроку  $i$ . Такий підхід дозволяє моделі проєктувати ознаки в різні підпростори та успішно адаптуватися до різних рівнів абстракції попри спільні вагові матриці  $A, B, C, D$  всередині самих блоків простору станів.

#### 2.2.4 Загальна архітектура

Загальна архітектура розробленої мережі представлена на рис. 2.3.



**Рисунок 2.3** – Архітектура гібридної нейронної мережі Bidirectional Shared Mamba

### 2.2.5 Формулювання функцій втрат

Процес оптимізації гібридної мережі керується складеною цільовою функцією втрат, яка балансує піксельну точність відтворення кольору, суворе дотримання підказок користувача та візуальний реалізм згенерованого зображення.

Загальна функція втрат  $\mathcal{L}_{\text{total}}$  формується як зважена сума трьох незалежних компонентів:

$$\mathcal{L}_{\text{total}} = \lambda_{L1} \mathcal{L}_{L1} + \lambda_{\text{hint}} \mathcal{L}_{\text{hint}} + \lambda_{\text{LPIPS}} \mathcal{L}_{\text{LPIPS}},$$

де  $\lambda_{L1}$ ,  $\lambda_{\text{hint}}$ ,  $\lambda_{\text{LPIPS}}$  – відповідні вагові коефіцієнти, що задають пріоритет кожної метрики.

**Глобальна похибка кольоровості ( $\mathcal{L}_{L1}$ ).** Базовою метрикою для оцінки правильності прогнозування каналів  $a$  та  $b$  обрано середню абсолютну похибку. На відміну від MSE, яка схильна до генерації тьмяних та усереднених кольорів через надмірне штрафування значних відхилень,  $\mathcal{L}_{L1}$  стимулює модель до створення більш насичених результатів. Похибка обчислюється по всьому простору зображення між згенерованими  $\hat{Y} = (\hat{a}, \hat{b})$  та істинними  $Y = (a, b)$

каналами:

$$\mathcal{L}_{L1} = \frac{1}{N} \sum_{i=1}^N |Y_i - \hat{Y}_i|.$$

**Штраф за ігнорування підказок ( $\mathcal{L}_{\text{hint}}$ ).** Для забезпечення жорсткої реакції моделі на взаємодію з користувачем вводиться спеціалізований штраф, який обчислюється виключно в тих пікселях, де маска підказок  $M$  є активною. Щоб запобігти дисбалансу градієнтів між зображеннями з різною кількістю підказок, похибка нормалізується на фактичну кількість пікселів підказки у конкретному зображенні з додаванням константи  $\epsilon = 10^{-8}$  для уникнення ділення на нуль:

$$\mathcal{L}_{\text{hint}} = \frac{1}{\max(\epsilon, \sum M_i)} \sum_{i=1}^N M_i \cdot |Y_i - \hat{Y}_i|.$$

Цей компонент змушує нейромережу ставити найвищий пріоритет на бездоганне поширення цільового кольору саме з тих точок, які визначив користувач.

**Перцептивна похибка ( $\mathcal{L}_{\text{LPIPS}}$ ).** Оскільки піксельні метрики, такі як L1, часто не корелюють із суб'єктивним сприйняттям якості зображення людиною, до конвеєра навчання інтегровано перцептивну похибку LPIPS. Вона обчислює відстань між зображеннями у просторі високорівневих ознак попередньо навченої згорткової мережі, у нашій системі – архітектури VGG.

Оскільки VGG-екстрактор натренований на зображеннях у колірному просторі RGB, математичний граф обчислення похибки включає диференційовне перетворення колірних просторів «на льоту». Спочатку нормалізовані канали денормалізуються до своїх фізичних масштабів – до каналу  $L$  додається 1 і разом це масштабується коефіцієнтом 50.0, канали  $a, b$  – коефіцієнтом 110.0. Далі відновлений тензор CIELAB конвертується у простір RGB, обмежується у діапазоні  $[0, 1]$  та нормалізується до  $[-1, 1]$ :

$$\mathcal{L}_{\text{LPIPS}} = \text{VGG} \left( \text{Norm}(\text{LABtoRGB}(L, \hat{a}, \hat{b})), \text{Norm}(\text{LABtoRGB}(L, a, b)) \right).$$

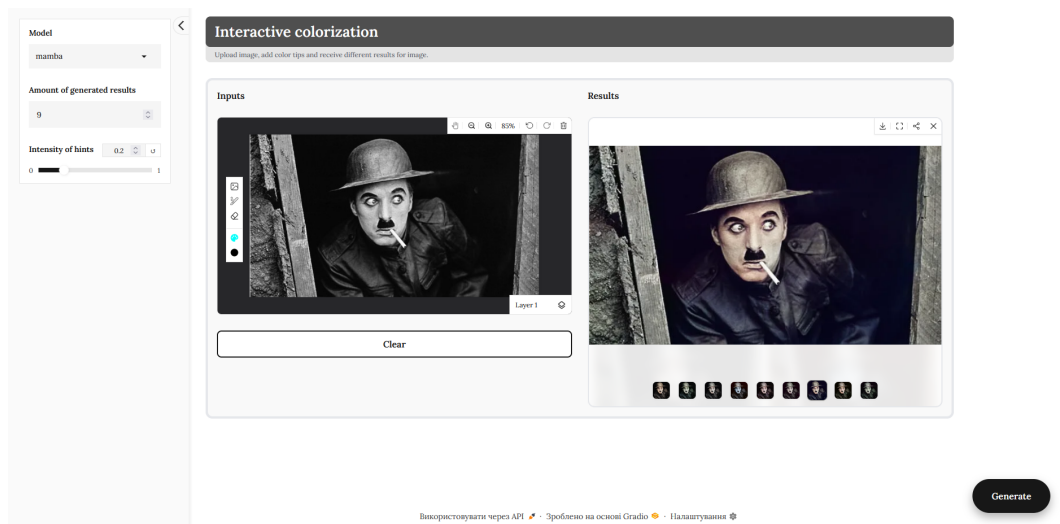
Завдяки використанню цієї функції втрат гібридна модель вчиться генерувати не лише математично близькі, але й візуально реалістичні та структуровані

текстури без ефекту неприродних градієнтних переходів.

## 2.3 Програмна реалізація та графічний інтерфейс користувача

Програмна реалізація розробленої системи виконана мовою програмування Python із використанням фреймворку глибокого навчання PyTorch. Це забезпечує ефективне паралельне виконання тензорних обчислень на графічних процесорах та мінімізує час генерації результату. Для побудови інтерактивного вебзастосунку було використано спеціалізований фреймворк Gradio.

Особливістю програмного бекенду є наявність розробленого диспетчера ресурсів ColorizationEngineAPI, який автоматично завантажує ваги нейромережі, відстежує зміни у конфігураційних файлах та здійснює примусове вивільнення відеопам'яті за допомогою збирача сміття під час перемикання між різними архітектурами. Це гарантує стабільну роботу системи навіть на комп'ютерах з обмеженими апаратними можливостями.



**Рисунок 2.4** – Інтерфейс розробленого застосунку для інтерактивної колоризації

Інтерфейс програми (рис. 2.4) реалізовано у вигляді інтерактивного вебзастосунку. Робочий процес алгоритмізовано за такими етапами:

1) **ініціалізація та налаштування:** користувач завантажує зображення, яке програмно перетворюється у простір LAB із виділенням каналу яскравості  $L$  як вхідної умови. Через бічну панель задаються гіперпараметри генерації: архітектура моделі, кількість вихідних варіацій та коефіцієнт глобальної інтенсивності підказок;

2) **введення просторових підказок:** за допомогою графічного інтерфейсу користувач наносить кольорові штрихи поверх чорно-білого зображення. Система фіксує ці мазки, вилучає їхні кольорові значення та формує розріджений тензор підказок разом із маскою інтенсивності підказок  $M$ ;

3) **інференс та візуалізація:** застосунок виконує прямий прохід обраної моделі в оптимізованому режимі `torch.inference_mode()`. Згенеровані канали  $a$  та  $b$  конкатенуються з вхідним каналом  $L$ , конвертуються у колірний простір RGB формату `uint8` та виводяться у блоці результатів як галерея багатоваріантних рішень.

Завдяки оптимізованій архітектурі моделі процес інференсу займає лічені мілісекунди.

## Висновки до розділу 2

У цьому розділі було детально описано процес інженерного проектування та практичної реалізації системи інтерактивної багатоваріантної колоризації. Розроблено оригінальну архітектуру Vi-Mamba, яка поєднує U-Net для контролю локальних меж та двонаправлену Mamba зі спільними вагами. Окрім того, створено зручний графічний застосунок.



### 3 ЕКСПЕРИМЕНТАЛЬНИЙ АНАЛІЗ РЕЗУЛЬТАТІВ ТА ОЦІНКА ЕФЕКТИВНОСТІ СИСТЕМИ

У цьому розділі наводяться результати комплексного тестування розробленої моделі B-Mamba та її порівняльний аналіз із передовими архітектурами в галузі колоризації зображень. Дослідження ефективності проведено як за допомогою об'єктивних математичних метрик PSNR, SSIM та MSE, так і за допомогою перцептивних генеративних критеріїв FID, KID та LPIPS.

Окрім аналізу якості генерованого зображення, особлива увага приділяється апаратній ефективності запропонованого методу – споживанню VRAM та кількості Time, що є критично важливими параметрами для систем інтерактивного розфарбовування реального часу.

#### 3.1 Апаратне забезпечення та параметри навчання

Процес навчання моделі здійснювався з використанням відеокарти NVIDIA GeForce RTX 5060 Ti та процесора AMD Ryzen 5 5600. Для забезпечення стабільності обчислень усі вхідні зображення масштабувалися до розміру  $256 \times 256$  пікселів, а розмір пакету (англ. batch size) становив 16 екземплярів.

Оптимізація вагових коефіцієнтів виконувалася за допомогою алгоритму AdamW із базовою швидкістю навчання  $1 \times 10^{-4}$  та коефіцієнтом згасання ваг (англ. weight decay)  $1 \times 10^{-4}$ . Для динамічного адаптивного керування кроком оптимізації застосовувався планувальник CosineAnnealingLR із мінімальною швидкістю навчання  $\eta_{\min} = 1 \times 10^{-6}$  та тривалістю циклу, що відповідає загальній кількості епох навчання.

Модель навчалася протягом 43 епох на тренувальній вибірці COCO 2017 Train, а валідація проводилася на COCO 2017 Val. Для розрахунку загальної цільової функції втрат було емпірично підібрано такі вагові коефіцієнти:

$\lambda_{L1} = 1.0$ ,  $\lambda_{\text{hint}} = 5.0$  та  $\lambda_{\text{LPIPS}} = 1.0$ .

Для прискорення навчання та зменшення споживання відеопам'яті застосовувалася змішана точність обчислень у форматі BFloat16 (англ. Brain Floating Point 16). На відміну від стандартного Float32, цей формат майже вдвічі скорочує обсяг пам'яті, необхідний для зберігання проміжних активацій, тоді як основні вагові коефіцієнти зберігаються у форматі Float32 для чисельної стабільності градієнтного спуску. Завдяки цьому тривалість однієї епохи скоротилася приблизно з 90 до 60 хвилин.

За підсумками процесу навчання значення функції втрат на тренувальній вибірці склало 0.1826. Під час контрольної перевірки значення функції втрат, усереднене для повністю автоматичної валідації та валідації з інтерактивними підказками, було на рівні 0.2236.

Така незначна розбіжність між тренувальними та валідаційними показниками підтверджує повну відсутність перенавчання та високу узагальнювальну здатність архітектури. Водночас динаміка похибки свідчить про те, що градієнтний спуск ще не досягнув абсолютного плато, вказуючи на наявність резерву для подальшого зниження похибки.

### 3.2 Методологія тестування та базові моделі

Для забезпечення об'єктивності результатів порівняння проводилося на комплексному наборі даних, в який увійшли такі датасети: COCO 2017 Test, Natural Color Dataset та Oxford-102 Flowers. Цей набір даних містить як зображення високої складності, наприклад заклад, який гармонійно поєднує в собі невелике кафе та крамницю свіжих овочів, так і зображення з усього одним об'єктом, наприклад яблуком. Усі моделі тестувалися в однакових апаратних умовах з ідентичною роздільною здатністю вхідних тензорів.

Для бенчмаркінгу було обрано три референсні моделі, які представляють різні етапи еволюції методів колоризації:

1) **ECCV16 [3]** – автоматична CNN, яка розв'язує задачу як мультимодальну класифікацію пікселів.

2) **SIGGRAPH17 [24]** – інтерактивна CNN, розроблена спеціально для підтримки користувацьких підказок. Вона слугує прямим конкурентом розробленій системі.

3) **Pix2Pix [6]** – інтерактивна cGAN, цілеспрямовано адаптована та навчена приймати просторові підказки як додаткову умову. Вона дозволяє оцінити ефективність класичного змагального підходу в задачах керованої генерації.

4) **DDColor [19]** – сучасна надважка автоматична модель на базі подвійних декодерів та просторової уваги, яка демонструє найвищу якість генерації серед автоматичного режиму обраних моделей.

### 3.3 Об'єктивний аналіз якості генерації та швидкодії

Основні результати кількісного оцінювання моделей показано в таблиці 3.1. Усі моделі отримали на вхід 3 підказки з найбільшою хроматичною магнітудою.

**Таблиця 3.1** – Основні результати моделей

| Модель                      | PSNR ↑       | SSIM ↑       | LPIPS ↓      | VRAM ↓           | Time ↓          |
|-----------------------------|--------------|--------------|--------------|------------------|-----------------|
| ECCV16 (Auto)               | 20.11        | 0.656        | 0.231        | <b>165.51 MB</b> | 6.05 ms         |
| DDColor (Auto)              | 19.94        | 0.653        | 0.199        | 1092.30 MB       | 35.37 ms        |
| Pix2Pix (Hinted, Our Impl.) | 19.51        | 0.610        | 0.246        | 241.28 MB        | <b>3.11 ms</b>  |
| SIGGRAPH17 (Hinted)         | 20.77        | 0.651        | 0.185        | 353.57 MB        | 15.53 ms        |
| <b>Ours (B-Mamba)</b>       | <b>22.53</b> | <b>0.664</b> | <b>0.143</b> | <b>187.29 MB</b> | <b>15.08 ms</b> |

Аналіз даних таблиці 3.1 свідчить про перевагу розробленої за більшістю критеріїв системи окрім VRAM та Time. Показник PSNR для B-Mamba склав 22.53 дБ, що на +2.42 дБ перевершує найближчого автоматичного конкурента ECCV16 та на +1.76 дБ перевершує інтерактивну модель SIGGRAPH17. Високий SSIM, який дорівнює 0.664, підтверджує, що архітектура U-Net бездоганно зберігає просторову геометрію та контури об'єктів. LPIPS, яка найкраще корелює зі сприйняттям людського ока, показала найкращий результат

– 0.143.

З точки зору обчислювальної ефективності розроблена гібридна модель продемонструвала видатні результати. Завдяки лінійній складності блоку Mamba та використанню спільних ваг, споживання VRAM під час інференсу склало лише 187.29 МБ. Для порівняння, потужна модель DDColor вимагає майже в 6 разів більше пам'яті – 1092.30 МБ – та працює у 2.3 рази повільніше – 35.37 мс проти 15.08 мс у B-Mamba. Це доводить високу придатність розробленої системи для розгортання на побутових пристроях.

### 3.4 Оцінка перцептивної реалістичності та точності керування

Додатково було проведено оцінку за допомогою FID, KID та MSE, зведених у таблиці 3.2.

**Таблиця 3.2** – Додаткові метрики якості генерації моделей

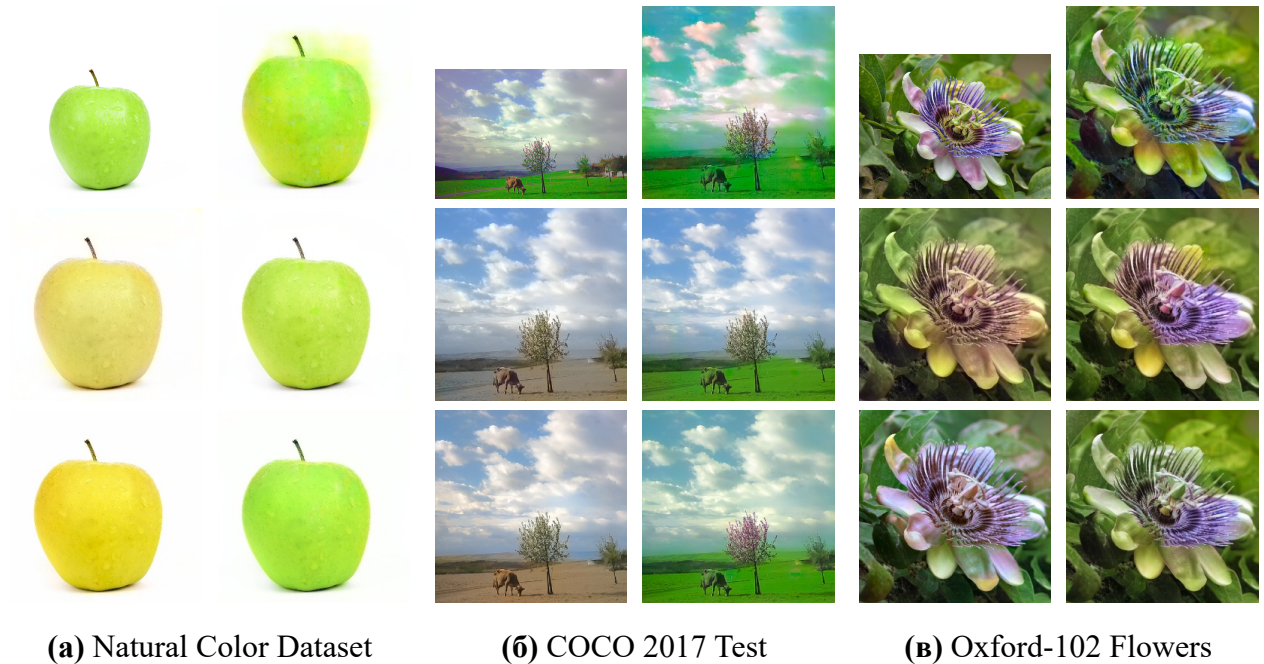
| Модель                      | FID ↓        | KID mean ↓     | KID std ↓      | MSE LAB ↓      | MSE HINT ↓                             |
|-----------------------------|--------------|----------------|----------------|----------------|----------------------------------------|
| ECCV16 (Auto)               | 23.000       | 0.01770        | 0.00720        | 0.02300        | 0.03807                                |
| DDColor (Auto)              | 5.031        | 0.00485        | 0.00848        | 0.02290        | 0.03773                                |
| Pix2Pix (Hinted, Our Impl.) | 8.437        | 0.00562        | 0.00842        | 0.02633        | 0.00039                                |
| SIGGRAPH17 (Hinted)         | 7.937        | 0.00452        | 0.00891        | 0.01889        | 0.00486                                |
| <b>Ours (B-Mamba)</b>       | <b>3.078</b> | <b>0.00313</b> | <b>0.00662</b> | <b>0.01332</b> | <b><math>6.1 \times 10^{-5}</math></b> |

Модель B-Mamba досягла досить низького значення FID – 3.078, порівняно з 5.031 у DDColor та 23.0 у ECCV16. Це, разом із найнижчим значенням  $KID_{mean}$ , яке дорівнює 0.00313, доводить, що кольорова палітра розробленої моделі є надзвичайно природною.

Критичним показником для розробленої інтерактивної системи є  $MSE_{HINT}$  – SIGGRAPH17 має 0.00486, що означає періодичне ігнорування або розмиття мережею вимог користувача. Натомість архітектура B-Mamba досягла  $6.1 \times 10^{-5}$ . Це беззаперечно доводить, що розроблена нейромережа ідеально слухається вводу оператора, жорстко фіксуючи колір у заданій точці та розумно поширюючи його на сусідні області без конфліктів у латентному просторі.

### 3.5 Візуальні результати

Окрім математичного оцінювання, було зібрано по одному результату з певного піддатасету для усіх моделей для порівняння візуально (Рис. 3.1)



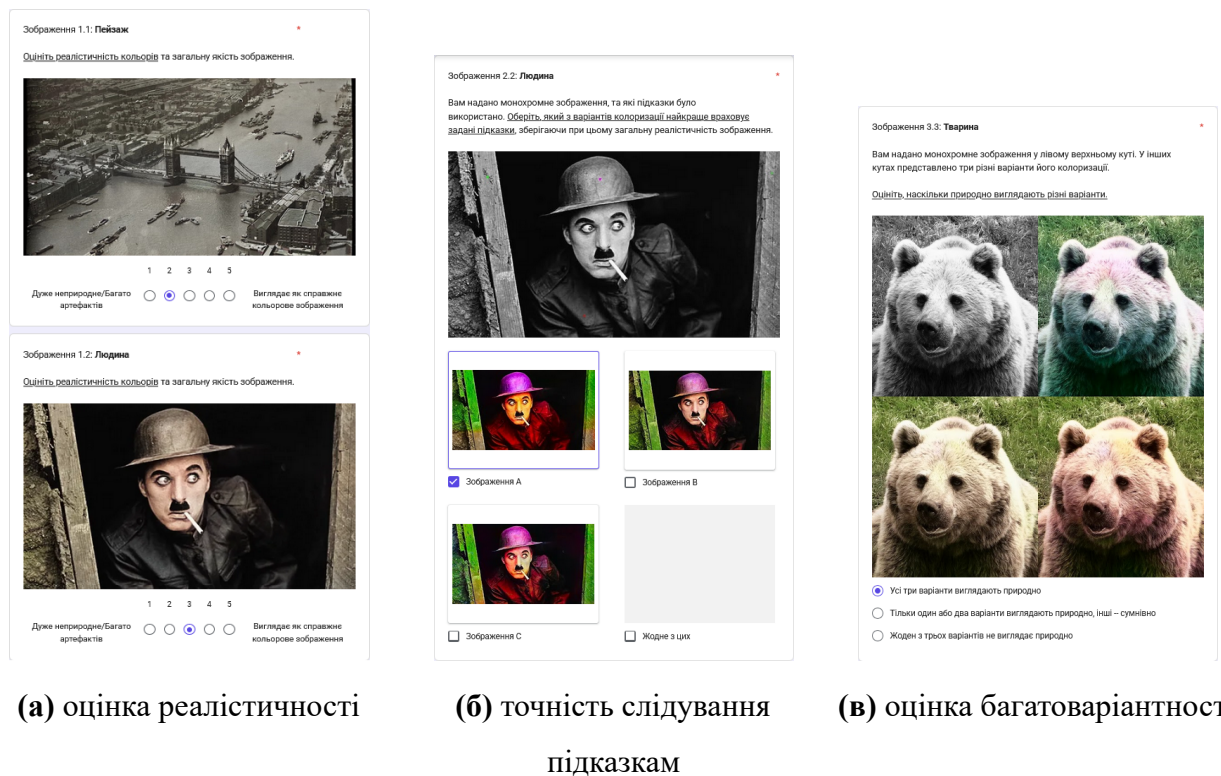
**Рисунок 3.1** – Візуальне порівняння результатів колоризації. У кожному блоці (зліва направо, зверху вниз): Оригінал, Pix2Pix, ECCV16, SIGGRAPH17, DDColor, B-Mamba.

### 3.6 Користувацьке дослідження якості колоризації

Для всебічної оцінки візуальної якості результатів роботи запропонованої гібридної моделі та її порівняння з базовим методом було проведено користувацьке дослідження. Об'єктивні метрики не завжди корелюють із тим, як людське око сприймає реалістичність кольорів, тому суб'єктивна оцінка є критично важливою для задач генерації зображень.

### 3.6.1 Методологія проведення опитування

Як виглядали етапи опитування можна побачити на рис. 3.2.



**Рисунок 3.2** – Інтерфейс користувацького дослідження

В опитуванні взяли участь 18 респондентів. Участь була повністю добровільною та анонімною, результати збиралися через платформу Google Forms. Для тестування було відібрано 6 категорій зображень різної складності: пейзаж, людина, тварина, кімната/інтер'єр, простий одиничний об'єкт та вулиця.

Опитування складалося з протоколу погодження та трьох послідовних етапів, кожен з яких був спрямований на оцінку конкретного аспекту роботи моделі:

1) протокол погодження: розповідається хто проводить дослідження, яка мета дослідження, які дані збираються, що відбувається під час дослідження, як використовуються та поширюються результати, який термін зберігання даних та інше.

2) оцінка реалістичності: респондентам демонстрували колоризоване зображення та пропонували оцінити його природність за 5-бальною шкалою (від 1 – «Дуже неприродне / Багато артефактів» до 5 – «Виглядає як справжнє кольорове зображення»);

3) точність слідування підказкам: респондентам демонстрували монохромне зображення із нанесеними колірними підказками та 3 варіанти колоризації від 3 різних моделей, а саме SIGGRAPH17, Pix2Pix та нашої запропонованої моделі. Завданням було обрати варіант, який найкраще враховує задані підказки, зберігаючи реалістичність;

4) оцінка багатоваріантності: респондентам демонстрували кілька згенерованих варіантів розфарбовування одного монохромного зображення. Завданням було оцінити, скільки з представлених варіантів виглядають природно.

### 3.6.2 Результати першого етапу: Оцінка реалістичності

На першому етапі середня оцінка реалістичності згенерованих зображень склала 3.509 балів з 5 можливих.



**Рисунок 3.3** – Розподіл відповідей респондентів щодо реалістичності автоматичних згенерованих зображень

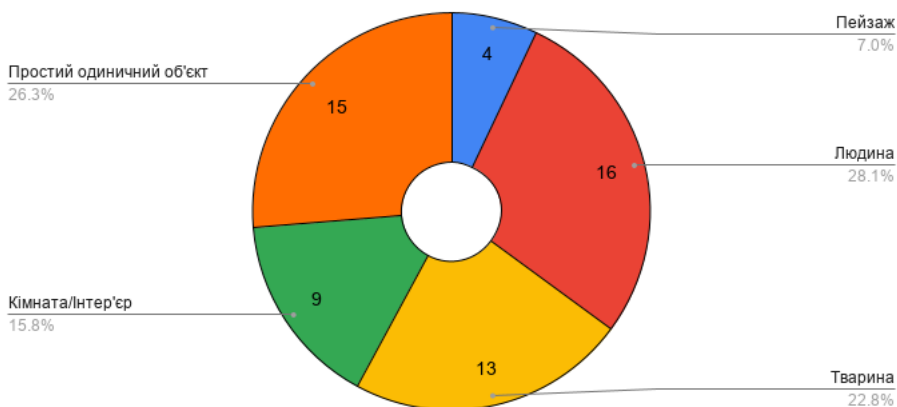
Найвищі оцінки отримали зображення в категоріях «простий одиничний об'єкт» та «людина», 4.056 та 3.944 відповідно, що свідчить про високу здатність моделі коректно відновлювати природні відтінки шкіри та локальні кольори ізольованих об'єктів.

Зображення з великою кількістю дрібних деталей або з висоти, такі категорії як «пейзаж» і «вулиця» отримали дещо нижчі бали, 2.333 та 3.556 відповідно, що пояснюється складністю просторового розподілу кольору в дрібних текстурах та падінням у безпечні сірі або сепійні кольори через невідоме зображення.

### 3.6.3 Результати другого етапу: Точність слідування підказкам

Цей етап мав на меті порівняти ефективність механізму інтерактивних підказок запропонованої моделі з альтернативними варіантами.

Розподіл відповідей респондентів щодо точності слідування підказкам



**Рисунок 3.4** – Розподіл відповідей респондентів щодо точності слідування підказкам

У 60.2% випадків респонденти обрали зображення, згенероване розробленою гібридною моделлю, як таке, що найкраще враховує задані підказки без втрати загальної якості зображення.

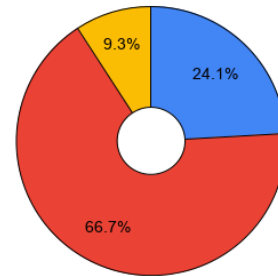


### 3.6.4 Результати третього етапу: Оцінка багатоваріантності

На фінальному етапі оцінювалася здатність моделі генерувати правдоподібні альтернативні варіанти.

Розподіл відповідей респондентів щодо правдоподібності альтернативних варіантів

- Жоден з трьох варіантів не виглядає природно
- Тільки один або два варіанти виглядають природно, інші – сумнівно
- Усі три варіанти виглядають природно



**Рисунок 3.5** – Розподіл відповідей респондентів щодо правдоподібності альтернативних варіантів

У 9.3% відповідей респонденти зазначили, що «Усі три варіанти виглядають природно». У 66.7% випадків користувачі обрали варіант «Тільки один або два варіанти виглядають природно, інші – сумнівно».

Цей результат підтверджує, що додавання генерації випадкових підказок дозволяє створювати різні версії розфарбовування, але зберігати їх візуальну правдоподібність в очах людини-спостерігача можливо, якщо колір цієї підказки теж буде генеруватися моделю, а не просто випадковим чином в нормальному розподілі.

### 3.6.5 Висновки за результатами користувацького дослідження

Проведене опитування підтвердило гіпотезу про те, що запропонована гібридна архітектура ефективно справляється із задачею інтерактивної багатоваріантної колоризації. Модель отримала високі суб'єктивні оцінки щодо загальної реалістичності, продемонструвала значну перевагу над базовим підходом у точності обробки локальних колірних підказок користувача, а також

довела здатність генерувати серію правдоподібних альтернативних кольорових рішень для одного зображення.

### **Висновки до розділу 3**

В ході експериментального дослідження розроблена нейромережева модель V-Mamba продемонструвала абсолютну перевагу над існуючими класичними аналогами ECCV16, SIGGRAPH17, Pix2Pix та перевершила сучасну важку архітектуру DDColor за перцептивними показниками якості генерації: FID 3.078, LPIPS 0.143. Водночас оптимізація через Shared Mamba дозволила скоротити використання VRAM до 187 МБ.

Рекордно низький  $MSE_{\text{HINT}} = 6.1 \times 10^{-5}$  підтверджує створення бездоганно керованої інтерактивної системи, яка здатна в реальному часі генерувати багатоваріантні результати за бажанням користувача.

## ВИСНОВКИ

У кваліфікаційній роботі розв'язано актуальну науково-прикладну задачу розробки обчислювально ефективною системи інтерактивної колоризації зображень із можливістю багатоваріантної генерації. За результатами проведених досліджень сформовано такі висновки:

1) У ході роботи було **повністю виконано завдання щодо аналізу** науково-технічної літератури. Дослідження існуючих методів комп'ютерного зору на базі CNN, GAN, Transformer та Diffusion показало, що традиційні моделі зводять задачу до детермінованого відображення, а багатоваріантні аналоги є занадто ресурсоемними для роботи в реальному часі. Це обґрунтувало необхідність проектування гібридної архітектури з використанням SSM.

2) Завдання з **підготовки даних та розробки алгоритму генерації підказок виконано в повному обсязі**. Сформовано комплексний набір даних для навчання та тестування на базі COCO 2017, Natural Color Dataset та Oxford-102 Flowers. Розроблено та програмно реалізовано алгоритм стохастичної генерації кольорних підказок на основі градієнтної значущості та семантичних центрів, що дозволило імітувати дії користувача та забезпечити багатоваріантність генерації без ускладнення архітектури мережі.

3) Успішно **спроєктовано та реалізовано гібридну нейромережеву архітектуру B-Mamba**, що поєднує локальну точність U-Net із глобальним рецептивним полем двонаправленої Mamba. Впровадження спільних ваг дозволило суттєво оптимізувати обчислювальні витрати. Для проведення порівняльного аналізу було успішно інтегровано базові моделі інших архітектурних класів, а саме ECCV16, SIGGRAPH17, Pix2Pix, DDColor.

4) Практичне завдання з **розробки програмного забезпечення виконано успішно**. Створено інтерактивний вебзастосунок із графічним інтерфейсом, який дозволяє користувачу в режимі реального часу керувати процесом колоризації за допомогою підказок.

5) Завдання щодо **навчання та комплексного оцінювання розробленої**

**моделі виконано.** За результатами об'єктивного тестування модель B-Mamba продемонструвала абсолютну перевагу над аналогами. Показник якості PSNR склав 22.53 дБ, що на 2.42 дБ перевищує результат автоматичної моделі ECCV16 та на 1.76 дБ – інтерактивної SIGGRAPH17. Архітектура забезпечила найкращу перцептивну якість із показниками LPIPS на рівні 0.143 та FID 3.078 (порівняно з 5.031 у важкої моделі DDColor). Система довела бездоганну точність дотримання користувацьких умов із рекордно низькою похибкою підказок  $MSE_{HINT}$  на рівні  $6.1 \times 10^{-5}$ . Підтверджено високу апаратну ефективність: час інференсу склав 15.08 мс при споживанні 187.29 МБ відеопам'яті, що в 2.3 раза швидше та майже в 6 разів економніше порівняно з еталонною моделлю DDColor.

**Перспективи подальших досліджень** у даній предметній області включають:

- теоретичне дослідження перспективних архітектур, зокрема аналіз можливостей інтеграції дифузійних процесів із SSM (Diffusion-Mamba) для підвищення якості генерації;
- удосконалення конвеєра багатоваріантності шляхом розробки допоміжної моделі для автоматичного передбачення природного кольору об'єктів;
- розширення набору базових моделей для проведення більш вичерпного та репрезентативного бенчмаркінгу;
- адаптацію розробленої архітектури для колоризації відеопотоку із забезпеченням часової узгодженості кадрів;
- розширення функціоналу програмного комплексу через впровадження додаткових інструментів для підвищення зручності користувацької взаємодії.

## ПЕРЕЛІК ПОСИЛАНЬ

- [1] Saeed Anwar et al. “Image Colorization: A Survey and Dataset”. English. In: *Information Fusion* 114 (Feb. 2025), p. 102720. ISSN: 1566-2535. DOI: 10.1016/j.inffus.2024.102720. URL: <https://doi.org/10.1016/j.inffus.2024.102720>.
- [2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. English. Cambridge, MA, USA: MIT Press, 2016. ISBN: 978-0262035613. URL: <http://www.deeplearningbook.org>.
- [3] Richard Zhang, Phillip Isola, and Alexei A. Efros. “Colorful Image Colorization”. English. In: *Computer Vision – ECCV 2016*. Ed. by Bastian Leibe et al. Cham: Springer International Publishing, 2016, pp. 649–666. ISBN: 978-3-319-46487-9. DOI: 10.1007/978-3-319-46487-9\_40. URL: [http://dx.doi.org/10.1007/978-3-319-46487-9\\_40](http://dx.doi.org/10.1007/978-3-319-46487-9_40).
- [4] Ling Yang et al. “Diffusion Models: A Comprehensive Survey of Methods and Applications”. English. In: *ACM Computing Surveys* 56.4 (Nov. 2024), pp. 1–39. ISSN: 0360-0300. DOI: 10.1145/3626235. URL: <https://doi.org/10.1145/3626235>.
- [5] Albert Gu and Tri Dao. “Mamba: Linear-Time Sequence Modeling with Selective State Spaces”. English. In: *arXiv preprint arXiv:2312.00752* (2023). arXiv: 2312.00752. URL: <https://arxiv.org/abs/2312.00752>.
- [6] Phillip Isola et al. “Image-To-Image Translation With Conditional Adversarial Networks”. English. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, July 2017, pp. 5967–5976. DOI: 10.1109/CVPR.2017.632. URL: <https://doi.org/10.48550/arXiv.1611.07004>.
- [7] Chitwan Saharia et al. “Palette: Image-to-Image Diffusion Models”. English. In: *ACM SIGGRAPH 2022 Conference Proceedings*. SIGGRAPH ’22. Vancouver, BC, Canada: Association for Computing Machinery, 2022. ISBN: 9781450393379. DOI: 10.1145/3528233.3530757. URL: <https://doi.org/10.1145/3528233.3530757>.

- [8] Yi Tay et al. “Long Range Arena: A Benchmark for Efficient Transformers”. English. In: *International Conference on Learning Representations (ICLR)*. 2021. URL: <https://arxiv.org/abs/2011.04006>.
- [9] Yi Tay et al. “Efficient Transformers: A Survey”. English. In: *ACM Computing Surveys (CSUR)* 55.6 (2022), pp. 1–28. DOI: 10.1145/3530811. URL: <https://arxiv.org/abs/2009.06732>.
- [10] Huiyu Zhai et al. “ColorMamba: Towards High-Quality NIR-to-RGB Spectral Translation with Mamba”. English. In: *arXiv preprint arXiv:2408.08087* (2024). arXiv: 2408.08087. URL: <https://arxiv.org/abs/2408.08087>.
- [11] Abhinav Attri et al. “Histogram Assisted Quality Aware Generative Model for Resolution Invariant NIR Image Colorization”. English. In: *arXiv preprint arXiv:2601.01103* (2026). arXiv: 2601.01103. URL: <https://arxiv.org/abs/2601.01103>.
- [12] Qian Jiang et al. “MFmamba: A Multi-function Network for Panchromatic Image Resolution Restoration Based on State-Space Model”. English. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 40. 7. 2026, pp. 5406–5414. arXiv: 2511.18888. URL: <https://arxiv.org/abs/2511.18888>.
- [13] Gustav Larsson, Michael Maire, and Gregory Shakhnarovich. “Learning representations for automatic colorization”. English. In: *European conference on computer vision*. Vol. 9908. Lecture Notes in Computer Science. Springer. Cham, 2016, pp. 577–593. ISBN: 978-3-319-46492-3. DOI: 10.1007/978-3-319-46493-0\_35. URL: [https://doi.org/10.1007/978-3-319-46493-0\\_35](https://doi.org/10.1007/978-3-319-46493-0_35).
- [14] Federico Baldassarre, Diego González Morín, and Lucas Rodés-Guirao. “Deep koalarization: Image colorization using cnns and inception-resnet-v2”. English. In: *arXiv preprint arXiv:1712.03400* (Dec. 2017). Project: <https://github.com/baldassarreFe/deep-koalarization>. DOI: 10.48550/arXiv.1712.03400. URL: <https://doi.org/10.48550/arXiv.1712.03400>.

- [15] Patricia Vitoria, Lara Raad, and Coloma Ballester. “ChromaGAN: Adversarial Picture Colorization with Semantic Class Distribution”. English. In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. Mar. 2020, pp. 2434–2443. DOI: 10.1109/WACV45572.2020.9093389. URL: [https://openaccess.thecvf.com/content\\_WACV\\_2020/html/Vitoria\\_ChromaGAN\\_Adversarial\\_Picture\\_Colorization\\_with\\_Semantic\\_Class\\_Distribution\\_WACV\\_2020\\_paper.html](https://openaccess.thecvf.com/content_WACV_2020/html/Vitoria_ChromaGAN_Adversarial_Picture_Colorization_with_Semantic_Class_Distribution_WACV_2020_paper.html).
- [16] Jun-Yan Zhu et al. “Unpaired Image-To-Image Translation Using Cycle-Consistent Adversarial Networks”. English. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. IEEE, Oct. 2017, pp. 2242–2251. DOI: 10.1109/ICCV.2017.244. URL: <https://doi.org/10.48550/arXiv.1703.10593>.
- [17] Manoj Kumar, Dirk Weissenborn, and Nal Kalchbrenner. “Colorization transformer”. English. In: *arXiv preprint arXiv:2102.04432* (2021). DOI: 10.48550/arXiv.2102.04432. URL: <https://doi.org/10.48550/arXiv.2102.04432>.
- [18] Zhitong Huang, Nanxuan Zhao, and Jing Liao. “UniColor: A Unified Framework for Multi-Modal Colorization with Transformer”. English. In: *ACM Transactions on Graphics* 41.6 (Nov. 2022). ISSN: 0730-0301. DOI: 10.1145/3550454.3555471. URL: <https://doi.org/10.1145/3550454.3555471>.
- [19] Xiaoyang Kang et al. “DDColor: Towards Photo-Realistic Image Colorization via Dual Decoders”. English. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, Oct. 2023, pp. 328–338. DOI: 10.1109/ICCV51070.2023.00037. URL: [https://openaccess.thecvf.com/content/ICCV2023/html/Kang\\_DDColor\\_Towards\\_Photo-Realistic\\_Image\\_Colorization\\_via\\_Dual\\_Decoders\\_ICCV\\_2023\\_paper.html](https://openaccess.thecvf.com/content/ICCV2023/html/Kang_DDColor_Towards_Photo-Realistic_Image_Colorization_via_Dual_Decoders_ICCV_2023_paper.html).
- [20] Arpit Bansal et al. “Cold Diffusion: Inverting Arbitrary Image Transforms Without Noise”. English. In: *Advances in Neural Information Processing Systems*. Ed. by A. Oh et al. Vol. 36. Curran Associates, Inc., 2023, pp. 41259–41282. DOI: 10.48550/arXiv.2208.09392. URL: <https://doi.org/10.48550/arXiv.2208.09392>.

- [21] Mingming He et al. “Deep exemplar-based colorization”. English. In: *ACM Transactions on Graphics* 37.4 (July 2018). ISSN: 0730-0301. DOI: 10.1145/3197517.3201365. URL: <https://doi.org/10.1145/3197517.3201365>.
- [22] Zheng Chang et al. “L-CAD: Language-based Colorization with Any-level Descriptions using Diffusion Priors”. English. In: *Advances in Neural Information Processing Systems*. Ed. by A. Oh et al. Vol. 36. Curran Associates, Inc., 2023, pp. 77174–77186. DOI: 10.48550/arXiv.2305.15217. URL: [https://proceedings.neurips.cc/paper\\_files/paper/2023/file/f3bfbd65743e60c685a3845bd61ce15f-Paper-Conference.pdf](https://proceedings.neurips.cc/paper_files/paper/2023/file/f3bfbd65743e60c685a3845bd61ce15f-Paper-Conference.pdf).
- [23] Nir Zabari et al. “Diffusing Colors: Image Colorization with Text Guided Diffusion”. English. In: *SIGGRAPH Asia 2023 Conference Papers*. New York, NY, USA: Association for Computing Machinery, 2023, 61:1–61:11. ISBN: 9798400703157. DOI: 10.1145/3610548.3618180. URL: <https://doi.org/10.1145/3610548.3618180>.
- [24] Richard Zhang et al. “Real-time user-guided image colorization with learned deep priors”. English. In: *arXiv preprint arXiv:1705.02999* (2017). DOI: 10.48550/arXiv.1705.02999. URL: <https://doi.org/10.48550/arXiv.1705.02999>.
- [25] Jooyeol Yun et al. “iColoriT: Towards Propagating Local Hints to the Right Region in Interactive Colorization by Leveraging Vision Transformer”. English. In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. IEEE, Jan. 2023, pp. 1787–1796. DOI: 10.1109/WACV56688.2023.00183. URL: [https://openaccess.thecvf.com/content/WACV2023/html/Yun\\_iColoriT\\_Towards\\_Propagating\\_Local\\_Hints\\_to\\_the\\_Right\\_Region\\_in\\_WACV\\_2023\\_paper.html](https://openaccess.thecvf.com/content/WACV2023/html/Yun_iColoriT_Towards_Propagating_Local_Hints_to_the_Right_Region_in_WACV_2023_paper.html).
- [26] Zhexin Liang et al. “Control Color: Multimodal Diffusion-Based Interactive Image Colorization: Z. Liang et al.” English. In: *International Journal of Computer Vision* 133.11 (2025), pp. 7897–7923. DOI: 10.1007/s11263-025-02549-6. URL: <https://doi.org/10.1007/s11263-025-02549-6>.



- [27] Anat Levin, Dani Lischinski, and Yair Weiss. “Colorization using optimization”. English. In: *ACM Transactions on Graphics* 23.3 (2004), pp. 689–694. ISSN: 0730-0301. DOI: 10.1145/1015706.1015780. URL: <https://doi.org/10.1145/1015706.1015780>.
- [28] Erik Reinhard et al. “Color transfer between images”. English. In: *IEEE Computer graphics and applications* 21.5 (2001), pp. 34–41. DOI: 10.1109/38.946629. URL: <https://doi.org/10.1109/38.946629>.
- [29] Tomihisa Welsh, Michael Ashikhmin, and Klaus Mueller. “Transferring color to greyscale images”. English. In: *Proceedings of the 29th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*. New York, NY, USA: Association for Computing Machinery, 2002, pp. 277–280. DOI: 10.1145/566570.566576. URL: <https://doi.org/10.1145/566570.566576>.
- [30] Ze Liu et al. “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows”. English. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021, pp. 10012–10022. arXiv: 2103.14030. URL: <https://arxiv.org/abs/2103.14030>.
- [31] Lianghui Zhu et al. “Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model”. English. In: *arXiv preprint arXiv:2401.09417* (2024). arXiv: 2401.09417. URL: <https://arxiv.org/abs/2401.09417>.

## ДОДАТОК А ФРАГМЕНТИ ВИХІДНОГО КОДУ ПРОГРАМИ

У цьому додатку наведено фрагменти вихідного коду найбільш критичних модулів розробленої системи інтерактивної колоризації. Повний вихідний код проєкту, включно зі скриптами для навчання, валідації та графічним інтерфейсом користувача, доступний у публічному репозиторії за посиланням: <https://github.com/KlymentievMaksym/interactive-colorization-engine>.

### A.1 Реалізація двонаправленого блоку Shared Mamba

Нижче наведено реалізацію основного архітектурного блоку Bi-Mamba, що використовує механізм розділення ваг для прямого та зворотного сканування послідовності ознак.

#### A.1.1 Допоміжні модулі

Лістинг 1: Клас модуля Shared Mamba (PyTorch)

```

1 import torch.nn as nn
2 from mamba_ssm import Mamba
3
4 class MambaShared(nn.Module):
5     def __init__(
6         self,
7         d_model: int,
8         d_state: int = 64,
9         d_conv: int = 4,
10        expand: int = 2,
11        layers: int = 6,
12        blocks: int = 2
13    ):
14        super().__init__()
15        self.layers = layers
16
17        self.blocks = nn.ModuleList([
18            Mamba(d_model=d_model, d_state=d_state, d_conv=d_conv, expand=expand) for _ in range(blocks)
19        ])
20
21        self.norms = nn.ModuleList([nn.LayerNorm(d_model) for _ in range(layers)])
22
23    def forward(self, x):
24        expected_val_range = self.layers // len(self.blocks)
25
26        for i in range(self.layers):
27            idx_block = min(i // expected_val_range, len(self.blocks) - 1)
28            x = self.blocks[idx_block](self.norms[i](x)) + x
29
30        return x

```

Лістинг 2: Клас модуля DoubleConv (PyTorch)

```

1 import torch
2 import torch.nn as nn

```

```

3
4 class DoubleConv(nn.Module):
5     def __init__(self, in_channels: int, out_channels: int):
6         super().__init__()
7         self.conv = nn.Sequential(
8             nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1, bias=False),
9             nn.BatchNorm2d(out_channels),
10            nn.ReLU(inplace=True),
11            nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1, bias=False),
12            nn.BatchNorm2d(out_channels),
13            nn.ReLU(inplace=True)
14        )
15
16    def forward(self, x: torch.Tensor) -> torch.Tensor:
17        return self.conv(x)

```

## A.1.2 Основний модуль

Лістинг 3: Клас модуля MambaWrapper (PyTorch)

```

1 import torch
2 import torch.nn as nn
3
4 from colorization_engine.models.util_models import MambaShared, BaseColorizer
5 from colorization_engine.factory.registry import register_model
6
7 @register_model("mamba")
8 class MambaWrapper(BaseColorizer):
9     def __init__(self, d_model: int = 256, layers: int = 6, blocks: int = 2):
10         super().__init__()
11
12         # --- ENCODER ---
13         # Вхід: 1 канал L + 2 канали ab підказки() + 1 канал маски = 4 канали
14         self.enc1 = DoubleConv(4, 64)
15         self.pool1 = nn.Conv2d(64, 64, kernel_size=4, stride=2, padding=1) # H/2
16
17         self.enc2 = DoubleConv(64, 128)
18         self.pool2 = nn.Conv2d(128, 128, kernel_size=4, stride=2, padding=1) # H/4
19
20         self.enc3 = DoubleConv(128, d_model)
21         self.pool3 = nn.Conv2d(d_model, d_model, kernel_size=4, stride=2, padding=1) # H/8
22
23         # --- Mamba ---
24         self.mamba = MambaShared(d_model=d_model, layers=layers, blocks=blocks)
25
26         # --- DECODER ---
27         self.up3 = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=False)
28         self.dec3 = DoubleConv(d_model + d_model, 128) # Concat enc3
29
30         self.up2 = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=False)
31         self.dec2 = DoubleConv(128 + 128, 64) # Concat enc2
32
33         self.up1 = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=False)
34         self.dec1 = DoubleConv(64 + 64, 64) # Concat enc1
35
36         # --- FINAL LAYER ---
37         self.final_conv = nn.Sequential(
38             nn.Conv2d(64, 2, kernel_size=1),
39             nn.Tanh() # [-1, 1]
40         )
41
42     def forward(self, l_norm: torch.Tensor, hints: torch.Tensor | None = None) -> torch.Tensor:
43         """
44         l_norm: (B, 1, H, W)
45         hints: (B, 3, H, W) - канали [a, b, mask], згенеровані _receive_hints
46         """
47         batch_size, _, height, width = l_norm.shape
48         device = l_norm.device
49
50         if hints is None:
51             hints = torch.zeros((batch_size, 3, height, width), device=device)
52
53         x_in = torch.cat([l_norm, hints], dim=1)
54
55         # --- ENCODER FORWARD ---
56         feat1 = self.enc1(x_in) # (B, 64, H, W)
57         p1 = self.pool1(feat1) # (B, 64, H/2, W/2)

```

```

59     feat2 = self.enc2(p1)          # (B, 128, H/2, W/2)
60     p2 = self.pool2(feat2)        # (B, 128, H/4, W/4)
61
62     feat3 = self.enc3(p2)          # (B, 256, H/4, W/4)
63     p3 = self.pool3(feat3)        # (B, 256, H/8, W/8)
64
65     # --- MAMBA FORWARD ---
66     b, c, h, w = p3.shape
67     x_flat = p3.view(b, c, h * w).permute(0, 2, 1).contiguous() # [B, L, D]
68
69     x_fwd = self.mamba(x_flat)
70     x_flat_rev = torch.flip(x_flat, dims=[1]) # Перевертаємо послідовність справа наліво, знизу вверху
71     x_rev = self.mamba(x_flat_rev)
72     x_rev = torch.flip(x_rev, dims=[1])
73
74     x_mamba = (x_fwd + x_rev) / 2.0
75
76     x_res = x_mamba.permute(0, 2, 1).contiguous().view(b, c, h, w)
77
78     # --- DECODER FORWARD ---
79     d3 = self.up3(x_res)
80     d3 = self.dec3(torch.cat([d3, feat3], dim=1)) # Злиття Mamba + локальний контекст H/4
81
82     d2 = self.up2(d3)
83     d2 = self.dec2(torch.cat([d2, feat2], dim=1)) # Злиття H/2
84
85     d1 = self.up1(d2)
86     d1 = self.dec1(torch.cat([d1, feat1], dim=1)) # Злиття H
87
88     ab_channels = self.final_conv(d1)
89
90     return ab_channels

```

## A.2 Генерація Гауссівських підказок

Наступні два лістинг демонструють механізм перетворення точкових кліків користувача на м'які розподіли кольору під час формування вхідного тензора для тренування, валідації та тестування.

Лістинг 4: Функція генерації маски інтерактивності

```

1  import torch
2
3  def get_gaussian_patch_box(
4      size: int,
5      sigma: float | None = None,
6      device: str | torch.device = 'cpu'
7  ) -> torch.Tensor:
8      """Generate 2D Gauss kernel with max 1.0"""
9      if sigma is None:
10         sigma = size / 4.0
11         coords = torch.arange(size, dtype=torch.float32, device=device) - size // 2
12         g = torch.exp(-(coords**2) / (2 * sigma**2))
13         g = g / g.max()
14         return g.view(-1, 1) * g.view(1, -1)
15
16  def get_gaussian_patch_circle(
17      size: int,
18      sigma: float | None = None,
19      device: str | torch.device = 'cpu'
20  ) -> torch.Tensor:
21      """Generate 2D Radial Gauss kernel with max 1.0 and strict circular bounds"""
22      if sigma is None:
23         sigma = size / 4.0
24
25      pad = size // 2
26      coords = torch.arange(size, dtype=torch.float32, device=device) - pad
27
28      Y, X = torch.meshgrid(coords, coords, indexing="ij")
29      dist_sq = X**2 + Y**2
30
31      patch = torch.exp(-dist_sq / (2 * sigma**2))
32      edge_val = torch.exp(torch.tensor(-(pad**2) / (2 * sigma**2), device=device))
33      patch = patch - edge_val

```

```

34
35 patch[dist_sq > pad*2] = 0.0
36 patch = patch.clamp(min=0.0)
37
38 patch = patch / patch.max()
39
40 return patch

```

### Лістинг 5: Функція генерації Гауссівських підказок

```

1 import torch
2 from albumentations import Compose
3 import cv2
4 import numpy as np
5
6 from colorization_engine.utils.saliency import FastSaliencySampler
7 from colorization_engine.utils.patches import get_gaussian_patch_circle as get_gaussian_patch
8
9 saliency_sampler = FastSaliencySampler(blur_kernel_size=15, uniform_prior=0.15)
10
11 def _receive_hints(
12     ab_tensor: torch.Tensor,
13     l_tensor: torch.Tensor,
14     min_hint_size: int = 2,
15     max_hint_size: int = 16,
16     num_hints_val: int = 3,
17     patch_size_val: int = 15,
18     training: bool = True
19 ) -> torch.Tensor:
20     _, h, w = ab_tensor.shape
21     device = ab_tensor.device
22     mask = torch.zeros((1, h, w), dtype=torch.float32, device=device)
23
24     if training:
25         dist = torch.distributions.Geometric(probs=torch.tensor([0.2]))
26         num_hints = int(dist.sample().item())
27
28         points = saliency_sampler.sample_points(l_tensor, num_hints * 3) if num_hints > 0 else []
29
30         if points and num_hints > 0:
31             placed_points = []
32             min_dist_sq = (max_hint_size * 1.5) ** 2
33
34             for y, x in points:
35                 too_close = any((y - py)**2 + (x - px)**2 < min_dist_sq for py, px in placed_points)
36                 if too_close:
37                     continue
38
39             placed_points.append((y, x))
40
41             pad = torch.randint(min_hint_size, max_hint_size, (1,), device=device).item()
42             patch_size = pad * 2 + 1
43             base_patch = get_gaussian_patch(patch_size, device=device) # type: ignore
44
45             intensity = torch.rand(1, device=device).item() * 0.8 + 0.2
46             patch = base_patch * intensity
47
48             y1, y2 = max(0, y - pad), min(h, y + pad + 1)
49             x1, x2 = max(0, x - pad), min(w, x + pad + 1)
50
51             py1, py2 = max(0, pad - y), patch_size - max(0, y + pad + 1 - h)
52             px1, px2 = max(0, pad - x), patch_size - max(0, x + pad + 1 - w)
53
54             mask[0, y1:y2, x1:x2] = torch.maximum(mask[0, y1:y2, x1:x2], patch[py1:py2, px1:px2])
55
56             if len(placed_points) >= num_hints:
57                 break
58
59     else:
60         pad = patch_size_val // 2
61         inhibition_radius = patch_size_val * 2
62
63         base_patch = get_gaussian_patch(size=patch_size_val, device=device)
64
65         mag = ab_tensor.pow(2).sum(dim=0)
66
67         for _ in range(num_hints_val):
68             idx = mag.argmax().item()
69             y = idx // w
70             x = idx % w
71
72             y1, y2 = max(0, y - pad), min(h, y + pad + 1)
73             x1, x2 = max(0, x - pad), min(w, x + pad + 1)
74
75             py1, py2 = max(0, pad - y), patch_size_val - max(0, y + pad + 1 - h)
76             px1, px2 = max(0, pad - x), patch_size_val - max(0, x + pad + 1 - w)
77

```

```

78         mask[0, y1:y2, x1:x2] = torch.maximum(mask[0, y1:y2, x1:x2], base_patch[py1:py2, px1:px2])
79
80         iy1, iy2 = max(0, y - inhibition_radius), min(h, y + inhibition_radius)
81         ix1, ix2 = max(0, x - inhibition_radius), min(w, x + inhibition_radius)
82         mag[iy1:iy2, ix1:ix2] = -1.0
83
84     hint_ab = ab_tensor * mask
85     hints = torch.cat([hint_ab, mask], dim=0)
86     return hints

```

### А.3 Багатоваріантність

У цьому підрозділі наведено програмний код, який відповідає за формування множини альтернативних кольорових рішень для одного вхідного зображення. Перший лістинг містить класи для просторового семплювання координат нових підказок на основі градієнтної значущості та виділення семантичних центрів об'єктів. Другий лістинг демонструє їх інтеграцію у базовий клас моделі для ітеративної генерації варіантів колоризації.

Лістинг 6: Класи просторового семплювання координат (PyTorch, OpenCV)

```

1  import cv2
2  import numpy as np
3  import torch
4  import torch.nn.functional as F
5
6  class FastSaliencySampler:
7      """
8      Клас для генерації координат підказок на основі градієнтної значущості.
9      Оптимізовано для використання у PyTorch DataLoader (CPU) та на GPU.
10     """
11     def __init__(self, blur_kernel_size: int = 15, uniform_prior: float = 0.15):
12         self.kernel_size = blur_kernel_size
13         self.uniform_prior = uniform_prior
14
15         self.sobel_x = torch.tensor([
16             [-1., 0., 1.],
17             [-2., 0., 2.],
18             [-1., 0., 1.]
19         ], dtype=torch.float32).view(1, 1, 3, 3)
20
21         self.sobel_y = torch.tensor([
22             [-1., -2., -1.],
23             [ 0., 0., 0.],
24             [ 1., 2., 1.]
25         ], dtype=torch.float32).view(1, 1, 3, 3)
26
27     def get_pdf(self, l_tensor: torch.Tensor) -> torch.Tensor:
28         """Обчислює 2D функцію щільності ймовірності (PDF) для зображення."""
29         device = l_tensor.device
30         # [C, H, W] -> [1, 1, H, W]
31         x = l_tensor.unsqueeze(0) if l_tensor.dim() == 3 else l_tensor
32
33         sx = self.sobel_x.to(device)
34         sy = self.sobel_y.to(device)
35
36         gx = F.conv2d(x, sx, padding=1)
37         gy = F.conv2d(x, sy, padding=1)
38
39         magnitude = torch.sqrt(gx**2 + gy**2 + 1e-6)
40
41         padding = self.kernel_size // 2
42         saliency = F.avg_pool2d(magnitude, kernel_size=self.kernel_size, stride=1, padding=padding)
43         saliency = saliency.squeeze(0).squeeze(0) # [H, W]
44
45         saliency_norm = saliency / (saliency.sum() + 1e-8)
46         uniform_dist = torch.ones_like(saliency_norm) / saliency_norm.numel()
47
48         pdf = (1.0 - self.uniform_prior) * saliency_norm + self.uniform_prior * uniform_dist

```

```

49     return pdf
50
51 def sample_points(self, l_tensor: torch.Tensor, num_points: int) -> list[tuple[int, int]]:
52     """
53     Повертає список координат (y, x) на основі карти значущості.
54     """
55     if num_points <= 0:
56         return []
57
58     pdf = self.get_pdf(l_tensor)
59     h, w = pdf.shape
60
61     # [H, W] -> [H*W]
62     pdf_flat = pdf.view(-1)
63
64     sampled_indices = torch.multinomial(pdf_flat, num_points, replacement=False)
65
66     # Відновлюємо 2D координати
67     points = []
68     for idx in sampled_indices:
69         y = (idx // w).item()
70         x = (idx % w).item()
71         points.append((y, x))
72
73     return points
74
75 class SemanticCenterSampler:
76     """
77     Генерує координати підказок у центрах семантичних об'єктів' (Distance Transform).
78     Уникає розміщення нових точок в об'єктах', які вже мають підказки.
79     """
80     def __init__(self, tolerance: int = 15):
81         # tolerance для floodFill. Визначає однорідність "" об'єкта' різниці (яскравості 0-255).
82         self.tolerance = tolerance
83
84     def sample_points(self, l_tensor: torch.Tensor, num_points: int, occupied_mask: torch.Tensor) -> list[tuple[int, int]]:
85         """
86         l_tensor: [1, 1, H, W]
87         occupied_mask: [1, 1, H, W]
88         """
89         l_np = l_tensor.squeeze().cpu().numpy()
90         l_uint8 = ((l_np + 1.0) / 2.0 * 255.0).clip(0, 255).astype(np.uint8)
91         h, w = l_uint8.shape
92
93         v = np.median(l_uint8)
94         lower = int(max(0, (1.0 - 0.33) * v))
95         upper = int(min(255, (1.0 + 0.33) * v))
96         edges = cv2.Canny(l_uint8, lower, upper)
97
98         inv_edges = cv2.bitwise_not(edges)
99         dist_map = cv2.distanceTransform(inv_edges, cv2.DIST_L2, 5)
100
101         mask_ff = np.zeros((h + 2, w + 2), dtype=np.uint8)
102         occ_np = occupied_mask.squeeze().cpu().numpy()
103
104         ys, xs = np.where(occ_np > 0)
105         for y, x in zip(ys, xs):
106             if mask_ff[y + 1, x + 1] == 0:
107                 cv2.floodFill(
108                     l_uint8, mask_ff, (int(x), int(y)), 0,
109                     loDiff=self.tolerance, upDiff=self.tolerance,
110                     flags=4 | (255 << 8) | cv2.FLOODFILL_MASK_ONLY
111                 )
112
113         valid_dist_map = dist_map.copy()
114         valid_dist_map[mask_ff[1:-1, 1:-1] == 255] = 0
115
116         points = []
117         for _ in range(num_points):
118             _, max_val, _, max_loc = cv2.minMaxLoc(valid_dist_map)
119
120             # Якщо максимальна відстань до краю < 3 пікселів, великих порожніх об'єктів' не лишилось
121             if max_val < 3.0:
122                 break
123
124             x, y = max_loc
125             points.append((y, x))
126
127             # Заливаємо знайдений об'єкт', щоб він більше не брав участь у виборі
128             cv2.floodFill(
129                 l_uint8, mask_ff, (int(x), int(y)), 0,
130                 loDiff=self.tolerance, upDiff=self.tolerance,
131                 flags=4 | (255 << 8) | cv2.FLOODFILL_MASK_ONLY
132             )
133
134         valid_dist_map[mask_ff[1:-1, 1:-1] == 255] = 0
135
136     return points

```

## Лістинг 7: Методи стохастичної генерації варіантів колоризації

```

1 from abc import ABC, abstractmethod
2 import random
3 import torch
4 import torch.nn as nn
5
6 from colorization_engine.utils.patches import get_gaussian_patch_circle as get_gaussian_patch
7 from colorization_engine.utils.saliency import FastSaliencySampler, SemanticCenterSampler
8
9 SEMANTIC_SAMPLER = SemanticCenterSampler(tolerance=20)
10 SALIENCY_SAMPLER = FastSaliencySampler(blur_kernel_size=15, uniform_prior=0.15)
11
12 class BaseColorizer(nn.Module, ABC):
13     """
14     Абстрактний базовий клас для всіх моделей колоризації.
15     Забезпечує єдиний інтерфейс для інференсу та валідації.
16     """
17     @abstractmethod
18     def forward(self, l_channel: torch.Tensor, hints: torch.Tensor | None = None) -> torch.Tensor:
19         """
20         l_channel: Tensor форми [B, 1, H, W] [-1, 1] (L)
21         hints: Tensor форми [B, 3, H, W] (ab + mask) or None
22         Returns: Tensor форми [B, 2, H, W] [-1, 1] (ab)
23         """
24         pass
25
26     def _generate_random_color(self, device: torch.device = torch.device('cpu')) -> torch.Tensor:
27         """
28         Генерує випадкові кольори у просторі CIELAB з фокусом на природніприглушені/ тони.
29         """
30         # std_dev = 0.35
31         std_dev = 0.25 # контролює насиченість
32
33         color_ab = torch.randn((2, 1, 1), device=device) * std_dev # нормальний розподіл, де центр у 0
34
35         # warm_bias = torch.tensor([[[0.15]], [[0.20]]], device=device)
36         # color_ab = color_ab + warm_bias
37
38         color_ab = torch.clamp(color_ab, min=-1.0, max=1.0)
39
40         return color_ab
41
42     def _generate_random_gaussian_hints(self, l_channel: torch.Tensor, base_hints: torch.Tensor | None, num_hints: int = 5):
43         """
44         Генерує тензор випадкових підказок у семантичних центрах незайнятих об'єктів'.
45         """
46         B, _, H, W = l_channel.shape
47         device = l_channel.device
48
49         new_hints = torch.zeros((B, 3, H, W), device=device)
50
51         if base_hints is not None:
52             occupied = base_hints[:, 2:3, :, :] > 0.05
53         else:
54             occupied = torch.zeros((B, 1, H, W), dtype=torch.bool, device=device)
55
56         min_dist_sq = (max_hint_size * 1.5) ** 2
57
58         for b in range(B):
59             if num_hints <= 0:
60                 continue
61
62             points = SALIENCY_SAMPLER.sample_points(l_channel[b:b+1], num_hints * 3) if random.random() < 0.5 else SEMANTIC_SAMPLER.sample_points(l_channel[b:b+1], num_hints * 3)
63             # points = SALIENCY_SAMPLER.sample_points(l_channel[b:b+1], num_hints * 3) + SEMANTIC_SAMPLER.sample_points(l_channel[b:b+1], num_hints * 3)
64             # random.shuffle(points)
65
66             if not points:
67                 continue
68
69             placed_points = []
70
71             for y, x in points:
72                 if occupied[b, 0, y, x]:
73                     continue
74
75                 too_close = any((y - py)**2 + (x - px)**2 < min_dist_sq for py, px in placed_points)
76                 if too_close:
77                     continue
78
79                 placed_points.append((y, x))
80
81             pad = int(torch.randint(min_hint_size, max_hint_size, (1,)).item())
82             patch_size = pad * 2 + 1
83             base_patch = get_gaussian_patch(patch_size, device=device) # type: ignore
84
85             intensity = torch.rand(1, device=device).item() * 0.8 + 0.2
86             patch = base_patch * intensity
87
88             y1, y2 = max(0, y - pad), min(H, y + pad + 1)

```



```

89         x1, x2 = max(0, x - pad), min(W, x + pad + 1)
90
91         py1, py2 = max(0, pad - y), patch_size - max(0, y + pad + 1 - H)
92         px1, px2 = max(0, pad - x), patch_size - max(0, x + pad + 1 - W)
93
94         current_mask_patch = patch[py1:py2, px1:px2]
95         new_hints[b, 2, y1:y2, x1:x2] = torch.maximum(new_hints[b, 2, y1:y2, x1:x2], current_mask_patch)
96
97         random_color = self._generate_random_color(device=device)
98         colored_patch = random_color * current_mask_patch
99         new_hints[b, 0:2, y1:y2, x1:x2] = new_hints[b, 0:2, y1:y2, x1:x2] + colored_patch
100
101         if len(placed_points) >= num_hints:
102             break
103
104     return new_hints
105
106 @torch.no_grad()
107 def sample(self, l_channel: torch.Tensor, hints: torch.Tensor | None = None, num_samples: int = 5, color_intensity: fl
108     """
109     Генерує варіанти колоризації, додаючи випадкові кольорові плями в порожні зони.
110     """
111     self.eval()
112     variants = []
113
114     clean_pred = self(l_channel, hints)
115     variants.append(clean_pred)
116
117     for _ in range(num_samples - 1):
118         gen_hints = self._generate_random_gaussian_hints(l_channel=l_channel, base_hints=hints, min_hint_size=min_hint
119         gen_hints[:, :2, :, :] *= color_intensity
120         gen_hints[:, :2, :, :] = torch.clamp(gen_hints[:, :2, :, :], -1.0, 1.0)
121
122         if hints is not None:
123             user_color = hints[:, :2, :, :]
124             user_mask = hints[:, 2:3, :, :]
125
126             gen_color = gen_hints[:, :2, :, :]
127             gen_mask = gen_hints[:, 2:3, :, :]
128
129             combined_mask = torch.maximum(user_mask, gen_mask)
130
131             combined_color = user_color + gen_color
132             combined_color = torch.clamp(combined_color, -1.0, 1.0)
133             # is_user = user_mask > 0
134             # combined_color = torch.where(is_user, user_color, gen_color)
135
136             combined_hints = torch.cat([combined_color, combined_mask], dim=1)
137         else:
138             combined_hints = gen_hints
139
140         variant_pred = self(l_channel, combined_hints)
141         variants.append(variant_pred)
142
143     return variants

```

## A.4 Комбінована функція втрат

Наступний лістинг демонструє кастомну функцію втрат, яка оптимізує модель одночасно за трьома критеріями: попиксельна точність L1, перцептивна реалістичність LPIPS та точність виконання інтерактивних підказок користувача.

Лістинг 8: Реалізація гібридної функції втрат (PyTorch)

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 from torchmetrics.image import LearnedPerceptualImagePatchSimilarity
5
6 from colorization_engine.loss import BaseLoss
7 from colorization_engine.factory.registry import register_loss
8 import kornia
9
10

```

```

11 @register_loss("colorization")
12 class ColorizationLoss(BaseLoss):
13     def __init__(self, lpips_weight: float, l1_weight: float, hints_weight: float):
14         super().__init__()
15         self.lpips_weight = lpips_weight
16         self.l1_weight = l1_weight
17         self.hints_weight = hints_weight
18
19         self.l1_loss = nn.L1Loss()
20
21         _lpips_module = LearnedPerceptualImagePatchSimilarity(net_type="vgg", normalize=True)
22         self.lpips_net = _lpips_module.net
23         self.lpips_net.eval()
24         self.lpips_net.requires_grad_(False)
25
26     def forward(
27         self,
28         pred: torch.Tensor,
29         target: torch.Tensor,
30         l_channel: torch.Tensor,
31         hint_mask: torch.Tensor | None = None
32     ) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
33         loss_l1 = self.l1_loss(pred, target) * self.l1_weight
34
35         loss_hints = torch.zeros_like(loss_l1)
36         if hint_mask is not None and hint_mask.any():
37             loss_hints_raw = F.l1_loss(pred * hint_mask, target * hint_mask, reduction='none')
38
39             loss_hints_per_image = loss_hints_raw.sum(dim=[1, 2, 3])
40             hint_pixels_per_image = hint_mask.sum(dim=[1, 2, 3]).clamp(min=1e-8)
41
42             loss_hints_avg = loss_hints_per_image / hint_pixels_per_image
43             loss_hints = loss_hints_avg.mean() * self.hints_weight
44
45         l_unnorm = (l_channel + 1.0) * 50.0
46         ab_pred_unnorm = pred * 110.0
47         ab_target_unnorm = target * 110.0
48
49         lab_pred = torch.cat([l_unnorm, ab_pred_unnorm], dim=1)
50         lab_target = torch.cat([l_unnorm, ab_target_unnorm], dim=1)
51
52         rgb_pred = kornia.color.lab_to_rgb(lab_pred).clamp(0.0, 1.0)
53         rgb_target = kornia.color.lab_to_rgb(lab_target).clamp(0.0, 1.0)
54
55         rgb_pred_norm = rgb_pred * 2.0 - 1.0
56         rgb_target_norm = rgb_target * 2.0 - 1.0
57
58         loss_lpips = self.lpips_net(rgb_pred_norm, rgb_target_norm).mean() * self.lpips_weight
59
60         total_loss = loss_l1 + loss_hints + loss_lpips
61
62         return total_loss, {
63             "loss_total": total_loss.detach(),
64             "loss_l1": loss_l1.detach(),
65             "loss_hints": loss_hints.detach(),
66             "loss_lpips": loss_lpips.detach(),
67         }

```